

Fuerza bruta y backtracking

Conjuntos de soluciones, árbol de backtracking y podas

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

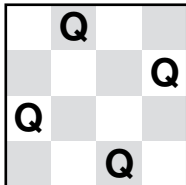
Agenda de hoy

- 1 ¿Por qué probar todo? Motivación y un poco de combinatoria.
- 2 Formalizar: soluciones candidatas, válidas, y qué nos piden de ellas.
- 3 Fuerza bruta.
- 4 Soluciones parciales y el árbol de backtracking.
- 5 La función recursiva y su semántica.
- 6 Podas por factibilidad.
- 7 Podas por optimalidad.
- 8 Variantes: construir, enumerar, contar.
- 9 Complejidad.
- 10 Tips + Conclusiones.

Un problema de 1848

Problema: n reinas

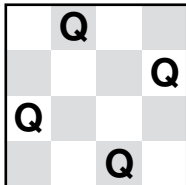
Ubicar n reinas en un tablero de ajedrez de $n \times n$ de manera que ninguna ataque a otra: no puede haber dos en la misma fila, columna ni diagonal.



Un problema de 1848

Problema: n reinas

Ubicar n reinas en un tablero de ajedrez de $n \times n$ de manera que ninguna ataque a otra: no puede haber dos en la misma fila, columna ni diagonal.

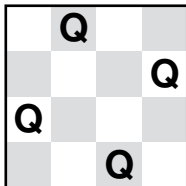


- Propuesto por Max Bezzel en 1848 para 8×8 ; generalizado a $n \times n$ en 1869.

Un problema de 1848

Problema: n reinas

Ubicar n reinas en un tablero de ajedrez de $n \times n$ de manera que ninguna ataque a otra: no puede haber dos en la misma fila, columna ni diagonal.

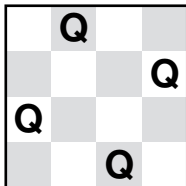


- Propuesto por Max Bezzel en 1848 para 8×8 ; generalizado a $n \times n$ en 1869.
- Gauss escribió en 1850 que uno puede convencerse de que hay 92 soluciones para $n = 8$ «*probando*, en una o un par de horas».

Un problema de 1848

Problema: n reinas

Ubicar n reinas en un tablero de ajedrez de $n \times n$ de manera que ninguna ataque a otra: no puede haber dos en la misma fila, columna ni diagonal.



- Propuesto por Max Bezzel en 1848 para 8×8 ; generalizado a $n \times n$ en 1869.
- Gauss escribió en 1850 que uno puede convencerse de que hay 92 soluciones para $n = 8$ «*probando*, en una o un par de horas».
- Hoy vamos a hacer eso: **probar**. Pero con método, y sabiendo cuánto cuesta.

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

- **Ocho casillas cualesquiera** del tablero: $\binom{64}{8} \approx 4,4 \times 10^9$.

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

- **Ocho casillas cualesquiera** del tablero: $\binom{64}{8} \approx 4,4 \times 10^9$.
- **Una reina por fila** (elegimos su columna): $8^8 \approx 1,7 \times 10^7$.

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

- **Ocho casillas cualesquiera** del tablero: $\binom{64}{8} \approx 4,4 \times 10^9$.
- **Una reina por fila** (elegimos su columna): $8^8 \approx 1,7 \times 10^7$.
- **Una reina por fila y por columna** (una permutación): $8! = 40\,320$.

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

- **Ocho casillas cualesquiera** del tablero: $\binom{64}{8} \approx 4,4 \times 10^9$.
- **Una reina por fila** (elegimos su columna): $8^8 \approx 1,7 \times 10^7$.
- **Una reina por fila y por columna** (una permutación): $8! = 40\,320$.

Es **el mismo problema** y las tres opciones contienen a las 92 soluciones. Pero el tercer universo es cien mil veces más chico que el primero.

¿Cuánto es «probar todo»?

Para $n = 8$, ¿cuántas configuraciones hay que probar? Depende de qué llamemos «configuración»:

- **Ocho casillas cualesquiera** del tablero: $\binom{64}{8} \approx 4,4 \times 10^9$.
- **Una reina por fila** (elegimos su columna): $8^8 \approx 1,7 \times 10^7$.
- **Una reina por fila y por columna** (una permutación): $8! = 40\,320$.

Es **el mismo problema** y las tres opciones contienen a las 92 soluciones. Pero el tercer universo es cien mil veces más chico que el primero.

Primera lección de hoy: antes de programar nada, hay que decidir *qué* es una solución candidata. Es una decisión de diseño, y cambia la complejidad.

¿Por qué probar todo?

- Para muchísimos problemas no se conoce nada sustancialmente mejor que probar todas las posibilidades: viajante de comercio, coloreo de grafos, satisfacibilidad, empaquetado. . .

¿Por qué probar todo?

- Para muchísimos problemas no se conoce nada sustancialmente mejor que probar todas las posibilidades: viajante de comercio, coloreo de grafos, satisfacibilidad, empaquetado. . .
- Aun cuando hay algo mejor (programación dinámica, greedy: las próximas clases), el algoritmo mejor casi siempre **nace** de la fuerza bruta: la misma recurrencia, con memoria; o la misma recurrencia, con una decisión que se puede tomar sin mirar el resto.

¿Por qué probar todo?

- Para muchísimos problemas no se conoce nada sustancialmente mejor que probar todas las posibilidades: viajante de comercio, coloreo de grafos, satisfacibilidad, empaquetado. . .
- Aun cuando hay algo mejor (programación dinámica, greedy: las próximas clases), el algoritmo mejor casi siempre **nace** de la fuerza bruta: la misma recurrencia, con memoria; o la misma recurrencia, con una decisión que se puede tomar sin mirar el resto.
- Y la fuerza bruta es el **oráculo** contra el que testeamos las versiones inteligentes.

¿Por qué probar todo?

- Para muchísimos problemas no se conoce nada sustancialmente mejor que probar todas las posibilidades: viajante de comercio, coloreo de grafos, satisfacibilidad, empaquetado. . .
- Aun cuando hay algo mejor (programación dinámica, greedy: las próximas clases), el algoritmo mejor casi siempre **nace** de la fuerza bruta: la misma recurrencia, con memoria; o la misma recurrencia, con una decisión que se puede tomar sin mirar el resto.
- Y la fuerza bruta es el **oráculo** contra el que testeamos las versiones inteligentes.

Lema de diseño (Erickson)

*Probar **todas** las posibilidades para la próxima decisión que sean consistentes con las decisiones anteriores, y dejar que la recursión se ocupe del resto. Sin ser inteligentes. Sin saltar opciones «obviamente» malas. Probar todo. Después lo hacemos más rápido.*

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

① ¿Cuántas **hojas** tiene un árbol binario completo de n niveles?

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

① ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos?

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos?

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.
- 4 ¿De cuántas formas puedo repartir n libros en k cajas distintas (cajas posiblemente vacías)?

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- ① ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- ② ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- ③ ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.
- ④ ¿De cuántas formas puedo repartir n libros en k cajas distintas (cajas posiblemente vacías)? k^n (cada libro elige su caja).

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.
- 4 ¿De cuántas formas puedo repartir n libros en k cajas distintas (cajas posiblemente vacías)? k^n (cada libro elige su caja).
- 5 ¿Cuántos subconjuntos de tamaño k tiene un conjunto de n elementos?

Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.
- 4 ¿De cuántas formas puedo repartir n libros en k cajas distintas (cajas posiblemente vacías)? k^n (cada libro elige su caja).
- 5 ¿Cuántos subconjuntos de tamaño k tiene un conjunto de n elementos? $\binom{n}{k}$.

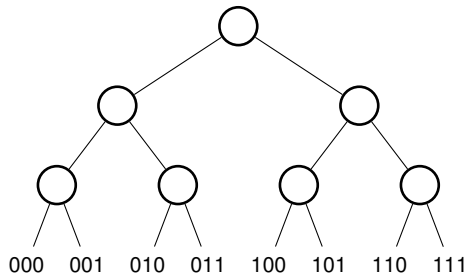
Quiz combinatorio

Para saber cuánto cuesta probar todo hay que saber contar. Sin mirar:

- 1 ¿Cuántas **hojas** tiene un árbol binario completo de n niveles? 2^n .
- 2 ¿Cuántos **subconjuntos** tiene un conjunto de n elementos? 2^n (cada elemento: está o no está).
- 3 ¿Cuántas **permutaciones** (ordenamientos sin repetir) tiene un conjunto de n elementos? $n!$.
- 4 ¿De cuántas formas puedo repartir n libros en k cajas distintas (cajas posiblemente vacías)? k^n (cada libro elige su caja).
- 5 ¿Cuántos subconjuntos de tamaño k tiene un conjunto de n elementos? $\binom{n}{k}$.

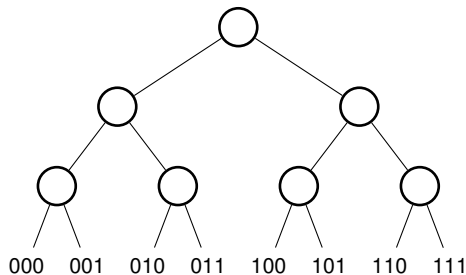
$n = 20$	n^2	2^n	$\binom{n}{n/2}$	$n!$
	400	$\approx 10^6$	$\approx 1,8 \times 10^5$	$\approx 2,4 \times 10^{18}$

Dos formas de ver 2^n

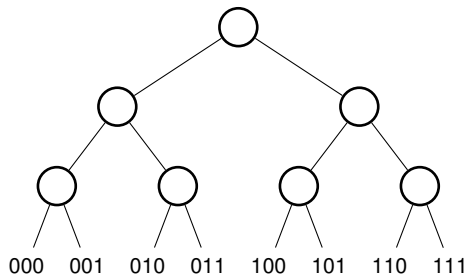


Dos formas de ver 2^n

- Un vector binario de longitud n es una **secuencia de n decisiones**:
 $a_1 \in \{0, 1\}$, luego $a_2, \dots$

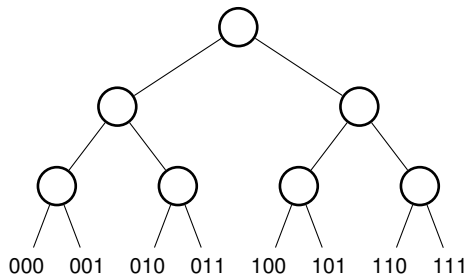


Dos formas de ver 2^n



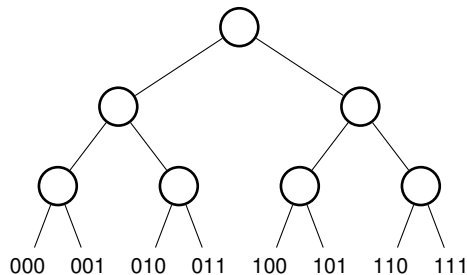
- Un vector binario de longitud n es una **secuencia de n decisiones**:
 $a_1 \in \{0, 1\}$, luego $a_2, \dots$
- Las hojas del árbol binario de n niveles **son** los 2^n vectores; el camino de la raíz a una hoja es el vector.

Dos formas de ver 2^n



- Un vector binario de longitud n es una **secuencia de n decisiones**:
 $a_1 \in \{0, 1\}$, luego $a_2, \dots$
- Las hojas del árbol binario de n niveles **son** los 2^n vectores; el camino de la raíz a una hoja es el vector.
- Un subconjunto de $\{1, \dots, n\}$ **es** un vector binario ($a_i = 1$ si i está).

Dos formas de ver 2^n



- Un vector binario de longitud n es una **secuencia de n decisiones**:
 $a_1 \in \{0, 1\}$, luego $a_2, \dots$
- Las hojas del árbol binario de n niveles **son** los 2^n vectores; el camino de la raíz a una hoja es el vector.
- Un subconjunto de $\{1, \dots, n\}$ **es** un vector binario ($a_i = 1$ si i está).

Este dibujo es, en esencia, todo lo que vamos a hacer hoy.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.
- 3 **Implementarla** con backtracking (esto no se demuestra: asumimos que el pasaje de la fórmula al código es directo).

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.
- 3 **Implementarla** con backtracking (esto no se demuestra: asumimos que el pasaje de la fórmula al código es directo).
- 4 Modificarla para que *construya* la solución, o las *enumere*, o las *cuenta*.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.
- 3 **Implementarla** con backtracking (esto no se demuestra: asumimos que el pasaje de la fórmula al código es directo).
- 4 Modificarla para que *construya* la solución, o las *enumere*, o las *cuenta*.
- 5 Proponer **podas** por factibilidad y por optimalidad, y demostrar que no pierden soluciones.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.
- 3 **Implementarla** con backtracking (esto no se demuestra: asumimos que el pasaje de la fórmula al código es directo).
- 4 Modificarla para que *construya* la solución, o las *enumere*, o las *cuenta*.
- 5 Proponer **podas** por factibilidad y por optimalidad, y demostrar que no pierden soluciones.
- 6 Calcular la **complejidad** temporal y espacial.

Qué tienen que saber hacer al terminar la práctica

- 1 Convertir un enunciado en lenguaje natural en una descripción de **conjuntos de soluciones**: candidatas, válidas, parciales; y para optimización, una función de valuación.
- 2 Dar una **función recursiva** que resuelva el problema, y demostrar que es correcta respecto de esa descripción.
- 3 **Implementarla** con backtracking (esto no se demuestra: asumimos que el pasaje de la fórmula al código es directo).
- 4 Modificarla para que *construya* la solución, o las *enumere*, o las *cuenta*.
- 5 Proponer **podas** por factibilidad y por optimalidad, y demostrar que no pierden soluciones.
- 6 Calcular la **complejidad** temporal y espacial.

Hoy: los seis, con énfasis en 1, 3, 4, 5 y 6. La clase que viene: el 2 y las demostraciones del 5, en profundidad.

Del enunciado a los conjuntos

Antes de escribir un algoritmo, traducimos el enunciado a tres cosas:

Del enunciado a los conjuntos

Antes de escribir un algoritmo, traducimos el enunciado a tres cosas:

- 1 Un conjunto Sols de **soluciones candidatas**: el *universo* de objetos que vamos a considerar. Tiene que ser un conjunto finito y con una descripción concreta (vectores, subconjuntos, permutaciones...).

Del enunciado a los conjuntos

Antes de escribir un algoritmo, traducimos el enunciado a tres cosas:

- 1 Un conjunto Sols de **soluciones candidatas**: el *universo* de objetos que vamos a considerar. Tiene que ser un conjunto finito y con una descripción concreta (vectores, subconjuntos, permutaciones...).
- 2 Un **predicado de validación** $válida(a)$, para $a \in \text{Sols}$, que dice si la candidata a es realmente una solución del problema.

Del enunciado a los conjuntos

Antes de escribir un algoritmo, traducimos el enunciado a tres cosas:

- 1 Un conjunto Sols de **soluciones candidatas**: el *universo* de objetos que vamos a considerar. Tiene que ser un conjunto finito y con una descripción concreta (vectores, subconjuntos, permutaciones...).
- 2 Un **predicado de validación** $válida(a)$, para $a \in \text{Sols}$, que dice si la candidata a es realmente una solución del problema.
- 3 El conjunto de **soluciones válidas**: $\text{Sols}_{\text{válidas}} := \{ a \in \text{Sols} : válida(a) \}$.

Del enunciado a los conjuntos

Antes de escribir un algoritmo, traducimos el enunciado a tres cosas:

- 1 Un conjunto Sols de **soluciones candidatas**: el *universo* de objetos que vamos a considerar. Tiene que ser un conjunto finito y con una descripción concreta (vectores, subconjuntos, permutaciones...).
- 2 Un **predicado de validación** $válida(a)$, para $a \in \text{Sols}$, que dice si la candidata a es realmente una solución del problema.
- 3 El conjunto de **soluciones válidas**: $\text{Sols}_{\text{válidas}} := \{ a \in \text{Sols} : válida(a) \}$.

Lo importante

El enunciado habla de reinas, mochilas o ciudades. Nuestros algoritmos van a hablar de **vectores**. Esta traducción es *nuestra* decisión, y hay que escribirla explícitamente.

Ejemplo: n reinas, tres traducciones

Sols

válida(a)

|Sols|

Ejemplo: n reinas, tres traducciones

Sols	válida(a)	Sols
Subconjuntos de n casillas del tablero de n^2 .	No hay dos casillas en la misma fila, columna ni diagonal.	$\binom{n^2}{n}$

Ejemplo: n reinas, tres traducciones

Sols	válida(a)	Sols
Subconjuntos de n casillas del tablero de n^2 .	No hay dos casillas en la misma fila, columna ni diagonal.	$\binom{n^2}{n}$
Vectores $a = (a_1, \dots, a_n)$ con $a_i \in \{1, \dots, n\}$: la reina de la fila i está en la columna a_i .	$a_i \neq a_j$ y $ a_i - a_j \neq i - j $ para todo $i < j$.	n^n

Ejemplo: n reinas, tres traducciones

Sols	válida(a)	Sols
Subconjuntos de n casillas del tablero de n^2 .	No hay dos casillas en la misma fila, columna ni diagonal.	$\binom{n^2}{n}$
Vectores $a = (a_1, \dots, a_n)$ con $a_i \in \{1, \dots, n\}$: la reina de la fila i está en la columna a_i .	$a_i \neq a_j$ y $ a_i - a_j \neq i - j $ para todo $i < j$.	n^n
Permutaciones a de $\{1, \dots, n\}$.	$ a_i - a_j \neq i - j $ para todo $i < j$.	$n!$

Ejemplo: n reinas, tres traducciones

Sols	válida(a)	Sols
Subconjuntos de n casillas del tablero de n^2 .	No hay dos casillas en la misma fila, columna ni diagonal.	$\binom{n^2}{n}$
Vectores $a = (a_1, \dots, a_n)$ con $a_i \in \{1, \dots, n\}$: la reina de la fila i está en la columna a_i .	$a_i \neq a_j$ y $ a_i - a_j \neq i - j $ para todo $i < j$.	n^n
Permutaciones a de $\{1, \dots, n\}$.	$ a_i - a_j \neq i - j $ para todo $i < j$.	$n!$

- Los tres Sols_{válidas} representan el **mismo** conjunto de tableros. ¿Por qué podemos restringirnos a permutaciones? Porque *toda* solución válida tiene exactamente una reina por fila y por columna (¡eso hay que argumentarlo!).

Ejemplo: n reinas, tres traducciones

Sols	válida(a)	Sols
Subconjuntos de n casillas del tablero de n^2 .	No hay dos casillas en la misma fila, columna ni diagonal.	$\binom{n^2}{n}$
Vectores $a = (a_1, \dots, a_n)$ con $a_i \in \{1, \dots, n\}$: la reina de la fila i está en la columna a_i .	$a_i \neq a_j$ y $ a_i - a_j \neq i - j $ para todo $i < j$.	n^n
Permutaciones a de $\{1, \dots, n\}$.	$ a_i - a_j \neq i - j $ para todo $i < j$.	$n!$

- Los tres Sols_{válidas} representan el **mismo** conjunto de tableros. ¿Por qué podemos restringirnos a permutaciones? Porque *toda* solución válida tiene exactamente una reina por fila y por columna (¡eso hay que argumentarlo!).
- Cuanto más «sabemos» de las soluciones válidas, más chico el universo... y más complicada la enumeración. Es un balance.

Cinco preguntas sobre el mismo conjunto

Una vez definido $Sols_{válidas}$, casi todo enunciado pregunta una de estas cosas:

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)
- **Construcción:** dar una. Algún $a \in \text{Sols}_{\text{válidas}}$, o «no hay».

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)
- **Construcción:** dar una. Algún $a \in \text{Sols}_{\text{válidas}}$, o «no hay».
- **Conteo:** ¿cuántas hay? $|\text{Sols}_{\text{válidas}}|$.

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)
- **Construcción:** dar una. Algún $a \in \text{Sols}_{\text{válidas}}$, o «no hay».
- **Conteo:** ¿cuántas hay? $|\text{Sols}_{\text{válidas}}|$.
- **Enumeración:** listarlas todas. $\text{Sols}_{\text{válidas}}$.

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)
- **Construcción:** dar una. Algún $a \in \text{Sols}_{\text{válidas}}$, o «no hay».
- **Conteo:** ¿cuántas hay? $|\text{Sols}_{\text{válidas}}|$.
- **Enumeración:** listarlas todas. $\text{Sols}_{\text{válidas}}$.
- **Optimización:** dar la mejor. Hace falta un **orden** $\preceq$ entre soluciones válidas, casi siempre dado por una **función de valuación** $\text{valor} : \text{Sols} \rightarrow \mathbb{R}$:

$$a \preceq b \iff \text{valor}(a) \leq \text{valor}(b).$$

Buscamos $a^* \in \text{Sols}_{\text{válidas}}$ tal que $\text{valor}(b) \leq \text{valor}(a^*)$ para todo $b \in \text{Sols}_{\text{válidas}}$ (o el valor $\text{valor}(a^*)$, o ambos).

Cinco preguntas sobre el mismo conjunto

Una vez definido $\text{Sols}_{\text{válidas}}$, casi todo enunciado pregunta una de estas cosas:

- **Decisión:** ¿existe alguna? $\exists a \in \text{Sols} : \text{válida}(a)$. (es un booleano)
- **Construcción:** dar una. Algún $a \in \text{Sols}_{\text{válidas}}$, o «no hay».
- **Conteo:** ¿cuántas hay? $|\text{Sols}_{\text{válidas}}|$.
- **Enumeración:** listarlas todas. $\text{Sols}_{\text{válidas}}$.
- **Optimización:** dar la mejor. Hace falta un **orden** $\preceq$ entre soluciones válidas, casi siempre dado por una **función de valuación** $\text{valor} : \text{Sols} \rightarrow \mathbb{R}$:

$$a \preceq b \iff \text{valor}(a) \leq \text{valor}(b).$$

Buscamos $a^* \in \text{Sols}_{\text{válidas}}$ tal que $\text{valor}(b) \leq \text{valor}(a^*)$ para todo $b \in \text{Sols}_{\text{válidas}}$ (o el valor $\text{valor}(a^*)$, o ambos).

Vamos a ver que el **mismo** algoritmo, con cambios mínimos, responde las cinco. Por eso la formalización es lo primero: el algoritmo es casi un subproducto.

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

- $\text{Sols} := \{0, 1\}^n$: el vector a representa el subconjunto $\{i : a_i = 1\}$.

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

- $\text{Sols} := \{0, 1\}^n$: el vector a representa el subconjunto $\{i : a_i = 1\}$.
- $\text{peso}(a) := \sum_{i=1}^n a_i w_i$ y $\text{valor}(a) := \sum_{i=1}^n a_i v_i$.

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

- $\text{Sols} := \{0, 1\}^n$: el vector a representa el subconjunto $\{i : a_i = 1\}$.
- $\text{peso}(a) := \sum_{i=1}^n a_i w_i$ y $\text{valor}(a) := \sum_{i=1}^n a_i v_i$.
- $\text{válida}(a) := [\text{peso}(a) \leq W]$.

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

- $\text{Sols} := \{0, 1\}^n$: el vector a representa el subconjunto $\{i : a_i = 1\}$.
- $\text{peso}(a) := \sum_{i=1}^n a_i w_i$ y $\text{valor}(a) := \sum_{i=1}^n a_i v_i$.
- $\text{válida}(a) := [\text{peso}(a) \leq W]$.
- Orden: $a \preceq b \iff \text{valor}(a) \leq \text{valor}(b)$. Piden $\max\{\text{valor}(a) : a \in \text{Sols}_{\text{válidas}}\}$ (y el a que lo logra).

Ejemplo: mochila 0/1 (el hilo conductor de hoy)

Problema: Mochila

Tenemos n objetos; el objeto i pesa $w_i \in \mathbb{N}$ y vale $v_i \in \mathbb{N}$. La mochila soporta un peso total W . Queremos elegir un subconjunto de objetos de valor total máximo cuyo peso total no supere W .

- $\text{Sols} := \{0, 1\}^n$: el vector a representa el subconjunto $\{i : a_i = 1\}$.
- $\text{peso}(a) := \sum_{i=1}^n a_i w_i$ y $\text{valor}(a) := \sum_{i=1}^n a_i v_i$.
- $\text{válida}(a) := [\text{peso}(a) \leq W]$.
- Orden: $a \preceq b \iff \text{valor}(a) \leq \text{valor}(b)$. Piden $\max\{\text{valor}(a) : a \in \text{Sols}_{\text{válidas}}\}$ (y el a que lo logra).

Notar que $\text{Sols}_{\text{válidas}} \neq \emptyset$ siempre: $(0, \dots, 0)$ es válida. Cuando el problema es de optimización, conviene saber si el conjunto sobre el que tomamos máximo puede ser vacío.

La instancia que vamos a usar toda la clase

$$n = 3, \quad w = (2, 3, 4), \quad v = (3, 4, 5), \quad W = 5.$$

a	$\text{peso}(a)$	$\text{valor}(a)$	válida	a	$\text{peso}(a)$	$\text{valor}(a)$	válida
$(0, 0, 0)$	0	0		$(1, 0, 0)$	2	3	
$(0, 0, 1)$	4	5		$(1, 0, 1)$	6	8	
$(0, 1, 0)$	3	4		$(1, 1, 0)$	5	7	
$(0, 1, 1)$	7	9		$(1, 1, 1)$	9	12	

La instancia que vamos a usar toda la clase

$$n = 3, \quad w = (2, 3, 4), \quad v = (3, 4, 5), \quad W = 5.$$

a	$\text{peso}(a)$	$\text{valor}(a)$	válida	a	$\text{peso}(a)$	$\text{valor}(a)$	válida
$(0, 0, 0)$	0	0	sí	$(1, 0, 0)$	2	3	sí
$(0, 0, 1)$	4	5	sí	$(1, 0, 1)$	6	8	no
$(0, 1, 0)$	3	4	sí	$(1, 1, 0)$	5	7	sí
$(0, 1, 1)$	7	9	no	$(1, 1, 1)$	9	12	no

La instancia que vamos a usar toda la clase

$$n = 3, \quad w = (2, 3, 4), \quad v = (3, 4, 5), \quad W = 5.$$

a	$\text{peso}(a)$	$\text{valor}(a)$	válida	a	$\text{peso}(a)$	$\text{valor}(a)$	válida
$(0, 0, 0)$	0	0	sí	$(1, 0, 0)$	2	3	sí
$(0, 0, 1)$	4	5	sí	$(1, 0, 1)$	6	8	no
$(0, 1, 0)$	3	4	sí	$(1, 1, 0)$	5	7	sí
$(0, 1, 1)$	7	9	no	$(1, 1, 1)$	9	12	no

- $|\text{Sols}| = 8, \quad |\text{Sols}_{\text{válidas}}| = 5, \quad \text{óptimo: } a^* = (1, 1, 0) \text{ con } \text{valor}(a^*) = 7.$

La instancia que vamos a usar toda la clase

$$n = 3, \quad w = (2, 3, 4), \quad v = (3, 4, 5), \quad W = 5.$$

a	peso(a)	valor(a)	válida	a	peso(a)	valor(a)	válida
(0, 0, 0)	0	0	sí	(1, 0, 0)	2	3	sí
(0, 0, 1)	4	5	sí	(1, 0, 1)	6	8	no
(0, 1, 0)	3	4	sí	(1, 1, 0)	5	7	sí
(0, 1, 1)	7	9	no	(1, 1, 1)	9	12	no

- $|\text{Sols}| = 8$, $|\text{Sols}_{\text{válidas}}| = 5$, óptimo: $a^* = (1, 1, 0)$ con $\text{valor}(a^*) = 7$.
- Esta tabla **es** un algoritmo de fuerza bruta, hecho a mano. Ahora lo hacemos a máquina.

Otro ejemplo: asignación de tareas

Problema: Asignación

Hay n personas y n tareas; asignarle la tarea j a la persona i cuesta $c_{ij} \in \mathbb{N}$. Cada persona hace exactamente una tarea y cada tarea la hace exactamente una persona. Minimizar el costo total.

Otro ejemplo: asignación de tareas

Problema: Asignación

Hay n personas y n tareas; asignarle la tarea j a la persona i cuesta $c_{ij} \in \mathbb{N}$. Cada persona hace exactamente una tarea y cada tarea la hace exactamente una persona. Minimizar el costo total.

- Sols := permutaciones π de $\{1, \dots, n\}$: la persona i hace la tarea $\pi(i)$.

Otro ejemplo: asignación de tareas

Problema: Asignación

Hay n personas y n tareas; asignarle la tarea j a la persona i cuesta $c_{ij} \in \mathbb{N}$. Cada persona hace exactamente una tarea y cada tarea la hace exactamente una persona. Minimizar el costo total.

- $\text{Sols} :=$ permutaciones π de $\{1, \dots, n\}$: la persona i hace la tarea $\pi(i)$.
- $\text{válida}(\pi) := \text{True}$. Elegimos un universo donde **todas** las candidatas son válidas: la restricción «una tarea por persona y viceversa» quedó absorbida en la definición de Sols.

Otro ejemplo: asignación de tareas

Problema: Asignación

Hay n personas y n tareas; asignarle la tarea j a la persona i cuesta $c_{ij} \in \mathbb{N}$. Cada persona hace exactamente una tarea y cada tarea la hace exactamente una persona. Minimizar el costo total.

- Sols := permutaciones π de $\{1, \dots, n\}$: la persona i hace la tarea $\pi(i)$.
- válida(π) := True. Elegimos un universo donde **todas** las candidatas son válidas: la restricción «una tarea por persona y viceversa» quedó absorbida en la definición de Sols.
- costo(π) := $\sum_{i=1}^n c_{i\pi(i)}$; $\pi \preceq \sigma \iff \text{costo}(\pi) \geq \text{costo}(\sigma)$ (minimizar es maximizar $-\text{costo}$).

Otro ejemplo: asignación de tareas

Problema: Asignación

Hay n personas y n tareas; asignarle la tarea j a la persona i cuesta $c_{ij} \in \mathbb{N}$. Cada persona hace exactamente una tarea y cada tarea la hace exactamente una persona. Minimizar el costo total.

- $\text{Sols} :=$ permutaciones π de $\{1, \dots, n\}$: la persona i hace la tarea $\pi(i)$.
- $\text{válida}(\pi) := \text{True}$. Elegimos un universo donde **todas** las candidatas son válidas: la restricción «una tarea por persona y viceversa» quedó absorbida en la definición de Sols.
- $\text{costo}(\pi) := \sum_{i=1}^n c_{i\pi(i)}$; $\pi \preceq \sigma \iff \text{costo}(\pi) \geq \text{costo}(\sigma)$ (minimizar es maximizar $-\text{costo}$).

Comparar con $\text{Sols}' :=$ vectores en $\{1, \dots, n\}^n$ y $\text{válida}(a) := [a \text{ es inyectiva}]$. Mismo Sols_{válidas}, $|\text{Sols}'| = n^n \gg n!$. Pero recorrer Sols' es «trivial» (un for anidado por coordenada) y recorrer las permutaciones requiere *recordar* qué tareas ya se usaron.

Ejercicio express: cuadrados latinos

Problema

Contar las grillas de $n \times n$ con valores en $\{1, \dots, n\}$ tales que ninguna fila ni columna repite un valor.

Dos minutos: propongan dos pares (Sols, válida) distintos, y para cada uno una candidata válida y una no válida (para $n = 2$).

Ejercicio express: cuadrados latinos

Problema

Contar las grillas de $n \times n$ con valores en $\{1, \dots, n\}$ tales que ninguna fila ni columna repite un valor.

Dos minutos: propongan dos pares (Sols, válida) distintos, y para cada uno una candidata válida y una no válida (para $n = 2$).

- $\text{Sols}_1 :=$ vectores de $\{1, \dots, n\}^{n^2}$ (la grilla leída por filas); $\text{válida}_1 :=$ ninguna fila ni columna repite.

Ejercicio express: cuadrados latinos

Problema

Contar las grillas de $n \times n$ con valores en $\{1, \dots, n\}$ tales que ninguna fila ni columna repite un valor.

Dos minutos: propongan dos pares (Sols, válida) distintos, y para cada uno una candidata válida y una no válida (para $n = 2$).

- $\text{Sols}_1 :=$ vectores de $\{1, \dots, n\}^{n^2}$ (la grilla leída por filas); $\text{válida}_1 :=$ ninguna fila ni columna repite.
- $\text{Sols}_2 :=$ vectores de n permutaciones de $\{1, \dots, n\}$ (una por fila); $\text{válida}_2 :=$ ninguna columna repite.

Ejercicio express: cuadrados latinos

Problema

Contar las grillas de $n \times n$ con valores en $\{1, \dots, n\}$ tales que ninguna fila ni columna repite un valor.

Dos minutos: propongan dos pares (Sols, válida) distintos, y para cada uno una candidata válida y una no válida (para $n = 2$).

- $\text{Sols}_1 :=$ vectores de $\{1, \dots, n\}^{n^2}$ (la grilla leída por filas); $\text{válida}_1 :=$ ninguna fila ni columna repite.
- $\text{Sols}_2 :=$ vectores de n permutaciones de $\{1, \dots, n\}$ (una por fila); $\text{válida}_2 :=$ ninguna columna repite.
- $|\text{Sols}_1| = n^{n^2}$ y $|\text{Sols}_2| = (n!)^n$. Para $n = 4$: $4,3 \times 10^9$ contra $3,3 \times 10^5$.

Fuerza bruta: el algoritmo genérico

Con Sols, válida y valor definidos, el algoritmo de fuerza bruta para optimización es:

```
procedure FuerzaBruta()  
  mejor  $\leftarrow -\infty$  ;  
  foreach  $a \in \text{Sols}$  do  
    if válida( $a$ ) then  
       $\text{mejor} \leftarrow \text{máx}(\text{mejor}, \text{valor}(a))$   
  return mejor
```

Fuerza bruta: el algoritmo genérico

Con Sols, válida y valor definidos, el algoritmo de fuerza bruta para optimización es:

```
procedure FuerzaBruta()  
  mejor  $\leftarrow -\infty$  ;  
  foreach  $a \in \text{Sols}$  do  
    if válida( $a$ ) then  
       $\text{mejor} \leftarrow \text{máx}(\text{mejor}, \text{valor}(a))$   
  return mejor
```

- Decisión: devolver *True* apenas una candidata es válida. Conteo: sumar. Enumeración: imprimir.

Fuerza bruta: el algoritmo genérico

Con Sols, válida y valor definidos, el algoritmo de fuerza bruta para optimización es:

```
procedure FuerzaBruta()  
   $mejor \leftarrow -\infty$  ;  
  foreach  $a \in \text{Sols}$  do  
    if  $válida(a)$  then  
       $mejor \leftarrow \text{máx}(mejor, \text{valor}(a))$   
  return  $mejor$ 
```

- Decisión: devolver *True* apenas una candidata es válida. Conteo: sumar. Enumeración: imprimir.
- Costo: $|\text{Sols}| \cdot (\text{costo de válida} + \text{costo de valor})$ **más** el costo de generar cada elemento de Sols.

Fuerza bruta: el algoritmo genérico

Con Sols, válida y valor definidos, el algoritmo de fuerza bruta para optimización es:

```
procedure FuerzaBruta()  
  mejor  $\leftarrow -\infty$  ;  
  foreach  $a \in \text{Sols}$  do  
    if válida( $a$ ) then  
       $\text{mejor} \leftarrow \text{máx}(\text{mejor}, \text{valor}(a))$   
  return mejor
```

- Decisión: devolver *True* apenas una candidata es válida. Conteo: sumar. Enumeración: imprimir.
- Costo: $|\text{Sols}| \cdot (\text{costo de válida} + \text{costo de valor})$ **más** el costo de generar cada elemento de Sols.
- Y ahí está el asunto: ¿cómo se hace ese *for each* sobre $\{0, 1\}^n$? ¿Sobre las permutaciones?

Generar $\{0, 1\}^n$: una decisión por vez

```
procedure Generar( $i, p$ )  
    //  $p = (a_1, \dots, a_{i-1})$  ya decidido  
    if  $i = n + 1$  then Procesar( $p$ ); return;  
    Generar( $i + 1, p \oplus 0$ );  
    Generar( $i + 1, p \oplus 1$ );
```

Se invoca Generar(1, ()). $\oplus$ es concatenar.

Generar $\{0, 1\}^n$: una decisión por vez

```
procedure Generar( $i, p$ )  
    //  $p = (a_1, \dots, a_{i-1})$  ya decidido  
    if  $i = n + 1$  then Procesar( $p$ ); return;  
    Generar( $i + 1, p \oplus 0$ );  
    Generar( $i + 1, p \oplus 1$ );
```

Se invoca Generar(1, ()). $\oplus$ es concatenar.

- Cada llamada **decide una coordenada**: la i -ésima. Las anteriores ya están fijas en p ; las siguientes las decide la recursión.

Generar $\{0, 1\}^n$: una decisión por vez

```
procedure Generar( $i, p$ )  
    //  $p = (a_1, \dots, a_{i-1})$  ya decidido  
    if  $i = n + 1$  then Procesar( $p$ ); return;  
    Generar( $i + 1, p \oplus 0$ );  
    Generar( $i + 1, p \oplus 1$ );
```

Se invoca Generar(1, ()). $\oplus$ es concatenar.

- Cada llamada **decide una coordenada**: la i -ésima. Las anteriores ya están fijas en p ; las siguientes las decide la recursión.
- Hay exactamente 2^n llamadas con $i = n + 1$, una por vector.

Generar $\{0, 1\}^n$: una decisión por vez

```
procedure Generar( $i, p$ )  
    //  $p = (a_1, \dots, a_{i-1})$  ya decidido  
    if  $i = n + 1$  then Procesar( $p$ ); return;  
    Generar( $i + 1, p \oplus 0$ );  
    Generar( $i + 1, p \oplus 1$ );
```

Se invoca Generar(1, ()). $\oplus$ es concatenar.

- Cada llamada **decide una coordenada**: la i -ésima. Las anteriores ya están fijas en p ; las siguientes las decide la recursión.
- Hay exactamente 2^n llamadas con $i = n + 1$, una por vector.
- Contar en binario de 0 a $2^n - 1$ también sirve, pero sólo para vectores binarios. La recursión sirve para **cualquier** universo definido por decisiones.

Mochila: fuerza bruta, versión 1

```
function mochila1( $i$ ,  $p$ )  
  if  $i = n + 1$  then  
    if peso( $p$ )  $\leq W$  then return valor( $p$ );  
    else return  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}_1(i + 1, p \oplus 0), \text{mochila}_1(i + 1, p \oplus 1))$  ;
```

Mochila: fuerza bruta, versión 1

```
function mochila1( $i$ ,  $p$ )  
  if  $i = n + 1$  then  
    if peso( $p$ )  $\leq W$  then return valor( $p$ );  
    else return  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}_1(i + 1, p \oplus 0), \text{mochila}_1(i + 1, p \oplus 1))$  ;
```

- mochila₁(1, ()) devuelve el óptimo. Es **exactamente** el for each de la fuerza bruta, con la generación hecha por recursión.

Mochila: fuerza bruta, versión 1

```
function mochila1( $i$ ,  $p$ )  
  if  $i = n + 1$  then  
    if peso( $p$ )  $\leq W$  then return valor( $p$ );  
    else return  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}_1(i + 1, p \oplus 0), \text{mochila}_1(i + 1, p \oplus 1))$  ;
```

- mochila₁(1, ()) devuelve el óptimo. Es **exactamente** el for each de la fuerza bruta, con la generación hecha por recursión.
- El $-\infty$ es el neutro del máximo: «esta hoja no aporta». Si todas las hojas devuelven $-\infty$, no hay solución válida (acá no pasa, $(0, \dots, 0)$ siempre lo es).

Mochila: fuerza bruta, versión 1

```
function mochila1( $i$ ,  $p$ )  
  if  $i = n + 1$  then  
    if peso( $p$ )  $\leq W$  then return valor( $p$ );  
    else return  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}_1(i + 1, p \oplus 0), \text{mochila}_1(i + 1, p \oplus 1))$  ;
```

- mochila₁(1, ()) devuelve el óptimo. Es **exactamente** el for each de la fuerza bruta, con la generación hecha por recursión.
- El $-\infty$ es el neutro del máximo: «esta hoja no aporta». Si todas las hojas devuelven $-\infty$, no hay solución válida (acá no pasa, $(0, \dots, 0)$ siempre lo es).
- Costo: 2^n hojas, cada una calcula peso(p) y valor(p) en $O(n)$: $O(n \cdot 2^n)$. Más el costo de concatenar (*copiar*) p en cada nodo, si lo copiamos.

¿Qué necesitamos recordar de las decisiones pasadas?

En la hoja sólo usamos $\text{peso}(p)$ y $\text{valor}(p)$: alcanza con **dos números** acumulados, pa y va , en lugar del vector p .

```
function mochila2( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return  $(pa \leq W) ? va : -\infty$ ;  
  return  
     $\text{máx}(\text{mochila}_2(i + 1, pa, va), \text{mochila}_2(i + 1, pa + w_i, va + v_i))$  ;
```

¿Qué necesitamos recordar de las decisiones pasadas?

En la hoja sólo usamos $\text{peso}(p)$ y $\text{valor}(p)$: alcanza con **dos números** acumulados, pa y va , en lugar del vector p .

```
function mochila2( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return ( $pa \leq W$ ) ?  $va$  :  $-\infty$ ;  
  return  
     $\text{máx}(\text{mochila}_2(i + 1, pa, va), \text{mochila}_2(i + 1, pa + w_i, va + v_i))$  ;
```

- Se invoca $\text{mochila}_2(1, 0, 0)$. Trabajo $O(1)$ por llamada, $2^{n+1} - 1$ llamadas: $O(2^n)$.

¿Qué necesitamos recordar de las decisiones pasadas?

En la hoja sólo usamos $\text{peso}(p)$ y $\text{valor}(p)$: alcanza con **dos números** acumulados, pa y va , en lugar del vector p .

```
function mochila2( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return ( $pa \leq W$ ) ?  $va$  :  $-\infty$ ;  
  return  
     $\text{máx}(\text{mochila}_2(i + 1, pa, va), \text{mochila}_2(i + 1, pa + w_i, va + v_i))$  ;
```

- Se invoca $\text{mochila}_2(1, 0, 0)$. Trabajo $O(1)$ por llamada, $2^{n+1} - 1$ llamadas: $O(2^n)$.
- Tuvimos que **generalizar el problema**: $\text{mochila}_2(i, pa, va)$ resuelve «la mejor forma de completar decisiones ya tomadas». El problema original es el caso particular $(1, 0, 0)$.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

- **Mochila:** i , peso acumulado, valor acumulado. Qué objetos exactamente elegimos es irrelevante para lo que falta.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

- **Mochila:** i , peso acumulado, valor acumulado. Qué objetos exactamente elegimos es irrelevante para lo que falta.
- **Asignación:** i (la próxima persona), el *conjunto* de tareas ya usadas, el costo acumulado.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

- **Mochila:** i , peso acumulado, valor acumulado. Qué objetos exactamente elegimos es irrelevante para lo que falta.
- **Asignación:** i (la próxima persona), el *conjunto* de tareas ya usadas, el costo acumulado.
- n **reinas:** i (la próxima fila) y *todas* las posiciones anteriores. Acá el resumen es la parcial entera.

Erickson: «el resumen de las decisiones pasadas»

Cada llamada toma **exactamente una** decisión, consistente con las anteriores. Para eso necesita: (1) la porción de la entrada que falta procesar (un índice i), y (2) un **resumen** de las decisiones ya tomadas, lo más chico posible.

- **Mochila:** i , peso acumulado, valor acumulado. Qué objetos exactamente elegimos es irrelevante para lo que falta.
- **Asignación:** i (la próxima persona), el *conjunto* de tareas ya usadas, el costo acumulado.
- n **reinas:** i (la próxima fila) y *todas* las posiciones anteriores. Acá el resumen es la parcial entera.

Regla práctica: los parámetros son *lo que la hoja necesita para decidir si es válida y cuánto vale*, más lo necesario para saber qué decisiones son legales.

Soluciones parciales y sucesoras

Soluciones parciales y sucesoras

Definición (para $\text{Sols} = \{0, 1\}^n$)

Una **solución parcial** es un vector $p = (a_1, \dots, a_{i-1})$ con $1 \leq i \leq n+1$: las primeras $i-1$ decisiones ya tomadas. Llamamos $\text{Sols}_{\text{parciales}}$ al conjunto de todas.

Si $i \leq n$, las **sucesoras** de p son $p \oplus 0$ y $p \oplus 1$ ($\oplus$: concatenar).

Las **extensiones** de p son las candidatas que empiezan con p :

$\text{Ext}(p) := \{ a \in \text{Sols} : a_j = p_j \text{ para } j < i \}.$

Soluciones parciales y sucesoras

Definición (para $\text{Sols} = \{0, 1\}^n$)

Una **solución parcial** es un vector $p = (a_1, \dots, a_{i-1})$ con $1 \leq i \leq n+1$: las primeras $i-1$ decisiones ya tomadas. Llamamos $\text{Sols}_{\text{parciales}}$ al conjunto de todas.

Si $i \leq n$, las **sucesoras** de p son $p \oplus 0$ y $p \oplus 1$ ($\oplus$: concatenar).

Las **extensiones** de p son las candidatas que empiezan con p :

$\text{Ext}(p) := \{a \in \text{Sols} : a_j = p_j \text{ para } j < i\}.$

- Las parciales de longitud n son las candidatas: $\text{Ext}(a) = \{a\}$. La vacía: $\text{Ext}() = \text{Sols}$.

Soluciones parciales y sucesoras

Definición (para $\text{Sols} = \{0, 1\}^n$)

Una **solución parcial** es un vector $p = (a_1, \dots, a_{i-1})$ con $1 \leq i \leq n+1$: las primeras $i-1$ decisiones ya tomadas. Llamamos $\text{Sols}_{\text{parciales}}$ al conjunto de todas.

Si $i \leq n$, las **sucesoras** de p son $p \oplus 0$ y $p \oplus 1$ ($\oplus$: concatenar).

Las **extensiones** de p son las candidatas que empiezan con p :

$\text{Ext}(p) := \{a \in \text{Sols} : a_j = p_j \text{ para } j < i\}.$

- Las parciales de longitud n son las candidatas: $\text{Ext}(a) = \{a\}$. La vacía: $\text{Ext}() = \text{Sols}$.
- Toda candidata a está en $\text{Ext}(p)$ para exactamente una parcial de cada longitud: sus prefijos.

Soluciones parciales y sucesoras

Definición (para $\text{Sols} = \{0, 1\}^n$)

Una **solución parcial** es un vector $p = (a_1, \dots, a_{i-1})$ con $1 \leq i \leq n+1$: las primeras $i-1$ decisiones ya tomadas. Llamamos $\text{Sols}_{\text{parciales}}$ al conjunto de todas.

Si $i \leq n$, las **sucesoras** de p son $p \oplus 0$ y $p \oplus 1$ ($\oplus$: concatenar).

Las **extensiones** de p son las candidatas que empiezan con p :

$\text{Ext}(p) := \{a \in \text{Sols} : a_j = p_j \text{ para } j < i\}.$

- Las parciales de longitud n son las candidatas: $\text{Ext}(a) = \{a\}$. La vacía: $\text{Ext}() = \text{Sols}$.
- Toda candidata a está en $\text{Ext}(p)$ para exactamente una parcial de cada longitud: sus prefijos.
- **Semántica:** p representa «ya decidí qué hago con los objetos $1, \dots, i-1$ ». Cada parcial es un *subproblema*.

Prefijos o sufijos: da lo mismo, pero elijan uno

- Hoy las parciales son **prefijos** $(a_1, \dots, a_{i-1})$ y decidimos a_i : la recursión avanza con $i + 1$.

Prefijos o sufijos: da lo mismo, pero elijan uno

- Hoy las parciales son **prefijos** $(a_1, \dots, a_{i-1})$ y decidimos a_i : la recursión avanza con $i + 1$.
- La práctica usa **sufijos** $(a_i, \dots, a_n)$ y decide a_{i-1} : la recursión avanza con $i - 1$, y el caso base es $i = 0$. Es el mismo árbol, espejado.

Prefijos o sufijos: da lo mismo, pero elijan uno

- Hoy las parciales son **prefijos** $(a_1, \dots, a_{i-1})$ y decidimos a_i : la recursión avanza con $i + 1$.
- La práctica usa **sufijos** $(a_i, \dots, a_n)$ y decide a_{i-1} : la recursión avanza con $i - 1$, y el caso base es $i = 0$. Es el mismo árbol, espejado.
- Con sufijos, el subproblema es «los objetos $1, \dots, i$ »: la fórmula queda $f(i, \dots)$ con $f(0, \dots)$ de base, que es cómodo para inducción en i .

Prefijos o sufijos: da lo mismo, pero elijan uno

- Hoy las parciales son **prefijos** $(a_1, \dots, a_{i-1})$ y decidimos a_i : la recursión avanza con $i + 1$.
- La práctica usa **sufijos** $(a_i, \dots, a_n)$ y decide a_{i-1} : la recursión avanza con $i - 1$, y el caso base es $i = 0$. Es el mismo árbol, espejado.
- Con sufijos, el subproblema es «los objetos $1, \dots, i$ »: la fórmula queda $f(i, \dots)$ con $f(0, \dots)$ de base, que es cómodo para inducción en i .

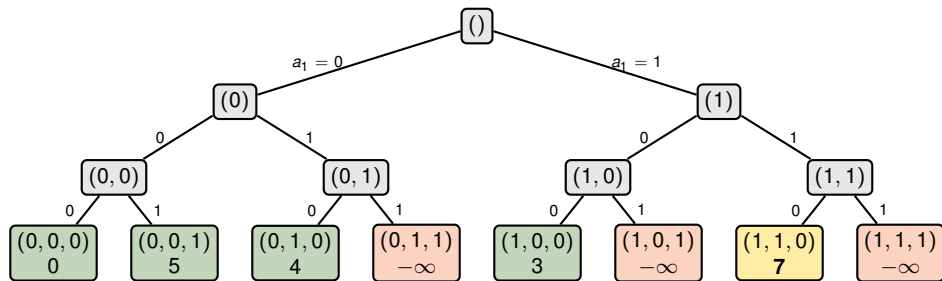
Lo que sí importa

Escribir explícitamente qué representa la parcial y qué representan los parámetros. « $f(i, pa, va)$ » no dice nada; «el máximo valor de una extensión válida cuando las decisiones $1, \dots, i - 1$ acumulan peso pa y valor va » sí.

El árbol de backtracking

Definición

El **árbol de backtracking** de un problema es el árbol con raíz cuyos nodos son las soluciones parciales, la raíz es $()$, y los hijos de p son sus sucesoras. Sus hojas son las candidatas.



Mochila con $w = (2, 3, 4)$, $v = (3, 4, 5)$, $W = 5$. Verde: válida; rojo: no válida; amarillo: óptima.

Árbol de backtracking = árbol de llamadas recursivas

- Cada nodo del árbol es una llamada a $\text{mochila}_2(i, pa, va)$; los parámetros son el *resumen* de la parcial. Cada arista es una llamada recursiva.

Árbol de backtracking = árbol de llamadas recursivas

- Cada nodo del árbol es una llamada a $\text{mochila}_2(i, pa, va)$; los parámetros son el *resumen* de la parcial. Cada arista es una llamada recursiva.
- El algoritmo lo recorre en **profundidad** (DFS): baja por $a_1 = 0$ hasta una hoja, **vuelve atrás** (*backtracks*) un nivel, prueba la otra rama, y así.

Árbol de backtracking = árbol de llamadas recursivas

- Cada nodo del árbol es una llamada a $\text{mochila}_2(i, pa, va)$; los parámetros son el *resumen* de la parcial. Cada arista es una llamada recursiva.
- El algoritmo lo recorre en **profundidad** (DFS): baja por $a_1 = 0$ hasta una hoja, **vuelve atrás** (*backtracks*) un nivel, prueba la otra rama, y así.
- Cada nodo devuelve hacia arriba el mejor valor de su subárbol; la raíz devuelve el mejor de todo el árbol. Es un máx «de abajo hacia arriba».

Árbol de backtracking = árbol de llamadas recursivas

- Cada nodo del árbol es una llamada a $mochila_2(i, pa, va)$; los parámetros son el *resumen* de la parcial. Cada arista es una llamada recursiva.
- El algoritmo lo recorre en **profundidad** (DFS): baja por $a_1 = 0$ hasta una hoja, **vuelve atrás** (*backtracks*) un nivel, prueba la otra rama, y así.
- Cada nodo devuelve hacia arriba el mejor valor de su subárbol; la raíz devuelve el mejor de todo el árbol. Es un máx «de abajo hacia arriba».

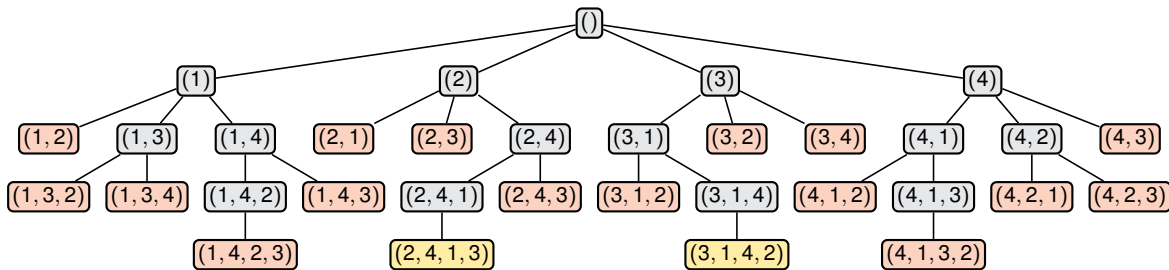
Tres objetos, una sola cosa

conjunto de soluciones parciales $\longleftrightarrow$ árbol de backtracking $\longleftrightarrow$ árbol de llamadas

Cuando dibujen el árbol de llamadas de una entrada concreta, tienen que poder señalar en cada nodo *qué parcial es*, y en cada hoja *qué candidata es* y si es válida.

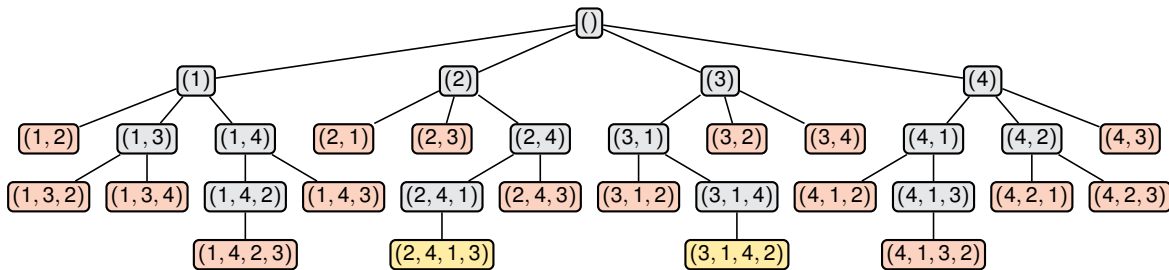
4 reinas: el árbol de llamadas con permutaciones

Sucesoras de p : $p \oplus j$ para cada columna j **no usada**. Al entrar a un nodo, si la reina nueva ataca a una anterior, se corta ahí (rojo):



4 reinas: el árbol de llamadas con permutaciones

Sucesoras de p : $p \oplus j$ para cada columna j **no usada**. Al entrar a un nodo, si la reina nueva ataca a una anterior, se corta ahí (rojo):



Rojo: ninguna extensión puede ser válida, no se generan sucesoras. Amarillo: las dos soluciones. 33 nodos visitados (16 rojos) contra 341 del árbol completo de $\{1, \dots, 4\}^4$. Ese ahorro se llama **poda**: después del café.

Antes del café: vocabulario

Sols	universo de candidatas ; lo elegimos nosotros.
válida, Sols _{válidas}	predicado de validación y conjunto de válidas .
valor, $\preceq$	valuación y orden, sólo en optimización.
Sols _{parciales} , sucesoras	parciales : decisiones tomadas; sucesoras: una decisión más.
Ext(p)	candidatas que extienden a p : el subárbol de p .
árbol de backtracking	parciales como nodos, sucesoras como hijos, candidatas como hojas.
resumen	los parámetros de la recursión: lo que necesito de las decisiones pasadas

Antes del café: vocabulario

Sols	universo de candidatas ; lo elegimos nosotros.
válida, Sols _{válidas}	predicado de validación y conjunto de válidas .
valor, $\preceq$	valuación y orden, sólo en optimización.
Sols _{parciales} , sucesoras	parciales : decisiones tomadas; sucesoras: una decisión más.
Ext(p)	candidatas que extienden a p : el subárbol de p .
árbol de backtracking	parciales como nodos, sucesoras como hijos, candidatas como hojas.
resumen	los parámetros de la recursión: lo que necesito de las decisiones pasadas

Después del café: cómo *no* recorrer todo el árbol, sin perder nada.

¿Descansamos 10 minutos?



La idea

En el árbol de 4 reinas cortamos en $(1, 2)$ porque las reinas de las filas 1 y 2 se atacan en diagonal: **ninguna** de las dos extensiones de $(1, 2)$ puede ser válida. ¿Para qué visitarlas?

La idea

En el árbol de 4 reinas cortamos en $(1, 2)$ porque las reinas de las filas 1 y 2 se atacan en diagonal: **ninguna** de las dos extensiones de $(1, 2)$ puede ser válida. ¿Para qué visitarlas?

Definición

Una **regla de factibilidad** es un predicado R sobre soluciones parciales tal que, para toda parcial p ,

$$\neg R(p) \implies \text{ninguna extensión de } p \text{ es válida, es decir } \text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset.$$

Podar por factibilidad es no explorar el subárbol de p cuando $\neg R(p)$.

La idea

En el árbol de 4 reinas cortamos en $(1, 2)$ porque las reinas de las filas 1 y 2 se atacan en diagonal: **ninguna** de las dos extensiones de $(1, 2)$ puede ser válida. ¿Para qué visitarlas?

Definición

Una **regla de factibilidad** es un predicado R sobre soluciones parciales tal que, para toda parcial p ,

$$\neg R(p) \implies \text{ninguna extensión de } p \text{ es válida, es decir } \text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset.$$

Podar por factibilidad es no explorar el subárbol de p cuando $\neg R(p)$.

Dicho al revés: $R(p)$ es una **condición necesaria** para que p tenga alguna extensión válida.

Qué hay que demostrar (y qué no)

- Lo que hay que demostrar para usar una poda: *si p no cumple R , entonces toda candidata $a \in \text{Ext}(p)$ no es válida*. Ni más ni menos.

Qué hay que demostrar (y qué no)

- Lo que hay que demostrar para usar una poda: *si p no cumple R , entonces toda candidata $a \in \text{Ext}(p)$ no es válida*. Ni más ni menos.
- Lo que **no** hace falta: que $R(p)$ garantice que *hay* una extensión válida. Una regla puede dejar pasar callejones sin salida; lo que no puede es cerrar un pasillo con salida.

Qué hay que demostrar (y qué no)

- Lo que hay que demostrar para usar una poda: *si p no cumple R , entonces toda candidata $a \in \text{Ext}(p)$ no es válida*. Ni más ni menos.
- Lo que **no** hace falta: que $R(p)$ garantice que *hay* una extensión válida. Una regla puede dejar pasar callejones sin salida; lo que no puede es cerrar un pasillo con salida.
- Tampoco hace falta que la poda *ayude*: una poda correcta que nunca corta es inútil, pero no es incorrecta.

Qué hay que demostrar (y qué no)

- Lo que hay que demostrar para usar una poda: *si p no cumple R , entonces toda candidata $a \in \text{Ext}(p)$ no es válida*. Ni más ni menos.
- Lo que **no** hace falta: que $R(p)$ garantice que *hay* una extensión válida. Una regla puede dejar pasar callejones sin salida; lo que no puede es cerrar un pasillo con salida.
- Tampoco hace falta que la poda *ayude*: una poda correcta que nunca corta es inútil, pero no es incorrecta.

Correcta es una propiedad matemática. *Útil* es una propiedad empírica.

Mochila: la poda obvia

Regla

$R(p) := [pa \leq W]$, donde pa es el peso acumulado por las decisiones de p .

Mochila: la poda obvia

Regla

$R(p) := [pa \leq W]$, donde pa es el peso acumulado por las decisiones de p .

- **Qué hay que probar:** si $pa > W$, toda extensión a de p cumple $\text{peso}(a) > W$.

Mochila: la poda obvia

Regla

$R(p) := [pa \leq W]$, donde pa es el peso acumulado por las decisiones de p .

- **Qué hay que probar:** si $pa > W$, toda extensión a de p cumple $\text{peso}(a) > W$.
- **Por qué es cierto:** $\text{peso}(a) = pa + \sum_{j \geq i} a_j w_j \geq pa$, porque los w_j son naturales. Se usa *una* propiedad de la entrada: los pesos no son negativos. Si hubiera pesos negativos, la poda sería **incorrecta**.

Mochila: la poda obvia

Regla

$R(p) := [pa \leq W]$, donde pa es el peso acumulado por las decisiones de p .

- **Qué hay que probar:** si $pa > W$, toda extensión a de p cumple $\text{peso}(a) > W$.
- **Por qué es cierto:** $\text{peso}(a) = pa + \sum_{j \geq i} a_j w_j \geq pa$, porque los w_j son naturales. Se usa *una* propiedad de la entrada: los pesos no son negativos. Si hubiera pesos negativos, la poda sería **incorrecta**.

→ Demostración en el pizarrón.

Mochila con la poda por factibilidad

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
  if  $pa > W$  then return  $-\infty$       // poda por factibilidad;  
  if  $i = n + 1$  then return  $va$ ;  
  return  
     $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

Mochila con la poda por factibilidad

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
    if  $pa > W$  then return  $-\infty$       // poda por factibilidad;  
    if  $i = n + 1$  then return  $va$ ;  
    return  
     $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

- La poda va **antes** del caso base: también corta hojas. En la hoja ya no hace falta chequear $pa \leq W$: si llegamos, la poda no cortó.

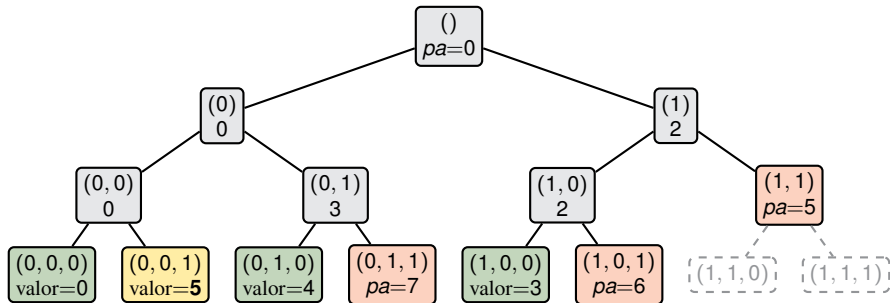
Mochila con la poda por factibilidad

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
    if  $pa > W$  then return  $-\infty$       // poda por factibilidad;  
    if  $i = n + 1$  then return  $va$ ;  
    return  
     $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

- La poda va **antes** del caso base: también corta hojas. En la hoja ya no hace falta chequear $pa \leq W$: si llegamos, la poda no cortó.
- Alternativa equivalente: no generar la sucesora $p \oplus 1$ cuando $pa + w_i > W$. Es la misma poda, un nivel más arriba.

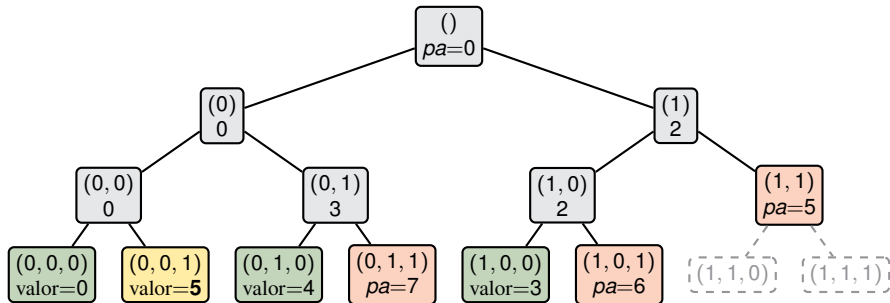
La poda depende de la instancia

Misma mochila, ahora con $W = 4$:



La poda depende de la instancia

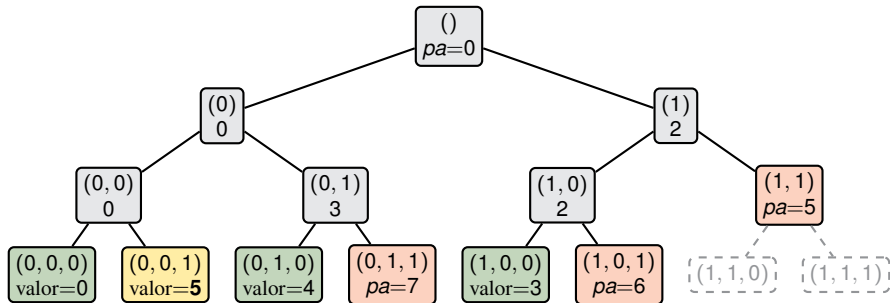
Misma mochila, ahora con $W = 4$:



- Con $W = 4$ la poda corta en $(1, 1)$ y ahorra dos nodos. Con $W = 5$ no corta nada por encima de las hojas. Con $W \geq 9$, no corta nunca.

La poda depende de la instancia

Misma mochila, ahora con $W = 4$:



- Con $W = 4$ la poda corta en $(1, 1)$ y ahorra dos nodos. Con $W = 5$ no corta nada por encima de las hojas. Con $W \geq 9$, no corta nunca.
- **Moraleja:** la poda *no cambia el peor caso*. Hay instancias (por ejemplo, $W \geq \sum w_i$) en que el árbol podado es el árbol entero.

n reinas: dos podas que son una

Con $\text{Sols} = \{1, \dots, n\}^n$ (una reina por fila, columna libre):

- $R_1(p) :=$ las reinas de p están en columnas distintas.

n reinas: dos podas que son una

Con $\text{Sols} = \{1, \dots, n\}^n$ (una reina por fila, columna libre):

- $R_1(p) :=$ las reinas de p están en columnas distintas.
- $R_2(p) :=$ ninguna pareja de reinas de p se ataca (columnas y diagonales).

n reinas: dos podas que son una

Con $\text{Sols} = \{1, \dots, n\}^n$ (una reina por fila, columna libre):

- $R_1(p) :=$ las reinas de p están en columnas distintas.
- $R_2(p) :=$ ninguna pareja de reinas de p se ataca (columnas y diagonales).
- **Demostración (para R_2 ; R_1 es un caso particular):** sea $p = (a_1, \dots, a_{i-1})$ con $\neg R_2(p)$: existen $j < k < i$ con $a_j = a_k$ o $|a_j - a_k| = |j - k|$. Sea $a \in \text{Ext}(p)$ cualquiera. Como a extiende a p , a_j y a_k son los mismos, luego ese par se ataca en a , luego $\neg \text{válida}(a)$. $\square$

n reinas: dos podas que son una

Con $\text{Sols} = \{1, \dots, n\}^n$ (una reina por fila, columna libre):

- $R_1(p) :=$ las reinas de p están en columnas distintas.
- $R_2(p) :=$ ninguna pareja de reinas de p se ataca (columnas y diagonales).
- **Demostración (para R_2 ; R_1 es un caso particular):** sea $p = (a_1, \dots, a_{i-1})$ con $\neg R_2(p)$: existen $j < k < i$ con $a_j = a_k$ o $|a_j - a_k| = |j - k|$. Sea $a \in \text{Ext}(p)$ cualquiera. Como a extiende a p , a_j y a_k son los mismos, luego ese par se ataca en a , luego $\neg \text{válida}(a)$. $\square$
- Elegir $\text{Sols} =$ permutaciones es lo mismo que tomar $\text{Sols} = \{1, \dots, n\}^n$ y podar con R_1 . **Restringir el universo y podar son la misma idea:** en un caso la restricción va en la definición de las sucesoras, en el otro en un *if*.

n reinas: dos podas que son una

Con $\text{Sols} = \{1, \dots, n\}^n$ (una reina por fila, columna libre):

- $R_1(p) :=$ las reinas de p están en columnas distintas.
- $R_2(p) :=$ ninguna pareja de reinas de p se ataca (columnas y diagonales).
- **Demostración (para R_2 ; R_1 es un caso particular):** sea $p = (a_1, \dots, a_{i-1})$ con $\neg R_2(p)$: existen $j < k < i$ con $a_j = a_k$ o $|a_j - a_k| = |j - k|$. Sea $a \in \text{Ext}(p)$ cualquiera. Como a extiende a p , a_j y a_k son los mismos, luego ese par se ataca en a , luego $\neg \text{válida}(a)$. $\square$
- Elegir $\text{Sols} =$ permutaciones es lo mismo que tomar $\text{Sols} = \{1, \dots, n\}^n$ y podar con R_1 . **Restringir el universo y podar son la misma idea:** en un caso la restricción va en la definición de las sucesoras, en el otro en un *if*.

En la práctica van a ver esta poda escrita así: «al agregar la reina i en la columna j , chequear contra las $i - 1$ anteriores». Cuesta $O(n)$ por nodo, contra el $O(n^2)$ de chequear la candidata entera en la hoja.

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

- 1 «Sea p una solución parcial tal que $\neg R(p)$.» Escribir qué significa concretamente $\neg R(p)$ (una suma que se pasó, un par que se ataca, un color repetido. . .).

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

- 1 «Sea p una solución parcial tal que $\neg R(p)$.» Escribir qué significa concretamente $\neg R(p)$ (una suma que se pasó, un par que se ataca, un color repetido. . .).
- 2 «Sea $a \in \text{Ext}(p)$ cualquiera.» Escribir qué sabemos de a : coincide con p en las primeras $i - 1$ coordenadas.

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

- 1 «Sea p una solución parcial tal que $\neg R(p)$.» Escribir qué significa concretamente $\neg R(p)$ (una suma que se pasó, un par que se ataca, un color repetido. . .).
- 2 «Sea $a \in \text{Ext}(p)$ cualquiera.» Escribir qué sabemos de a : coincide con p en las primeras $i - 1$ coordenadas.
- 3 Mostrar que **la violación se hereda**: lo que estaba mal en p sigue estando mal en a , porque a contiene a p y lo que agrega no lo arregla (monotonía: pesos ≥ 0 , una arista que ya estaba, etc.).

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

- 1 «Sea p una solución parcial tal que $\neg R(p)$.» Escribir qué significa concretamente $\neg R(p)$ (una suma que se pasó, un par que se ataca, un color repetido. . .).
- 2 «Sea $a \in \text{Ext}(p)$ cualquiera.» Escribir qué sabemos de a : coincide con p en las primeras $i - 1$ coordenadas.
- 3 Mostrar que **la violación se hereda**: lo que estaba mal en p sigue estando mal en a , porque a contiene a p y lo que agrega no lo arregla (monotonía: pesos ≥ 0 , una arista que ya estaba, etc.).
- 4 Concluir $\neg \text{válida}(a)$. Como a era cualquiera, $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset$.

El molde de la demostración

Casi todas las podas por factibilidad se demuestran igual. Conviene tener el molde:

- 1 «Sea p una solución parcial tal que $\neg R(p)$.» Escribir qué significa concretamente $\neg R(p)$ (una suma que se pasó, un par que se ataca, un color repetido. . .).
- 2 «Sea $a \in \text{Ext}(p)$ cualquiera.» Escribir qué sabemos de a : coincide con p en las primeras $i - 1$ coordenadas.
- 3 Mostrar que **la violación se hereda**: lo que estaba mal en p sigue estando mal en a , porque a contiene a p y lo que agrega no lo arregla (monotonía: pesos ≥ 0 , una arista que ya estaba, etc.).
- 4 Concluir $\neg \text{válida}(a)$. Como a era cualquiera, $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset$.

Cuidado

«Es obvio que si ya me pasé de peso, no puedo volver» **no** es una demostración. La frase que la reemplaza es: «como $w_j \geq 0$ para todo j , $\text{peso}(a) \geq pa > W$ ».

Lo que NO es una poda

- «Si el objeto i pesa más que el promedio, no lo pongo.» Puede perder el óptimo.
Es una **heurística**, no una poda.

Lo que NO es una poda

- «Si el objeto i pesa más que el promedio, no lo pongo.» Puede perder el óptimo. Es una **heurística**, no una poda.
- «En n reinas, la primera reina nunca va en la columna 1.» Falso para $n = 1$, y para $n = 4$: $(1, \dots)$ no da solución, pero para $n = 5$ sí: $(1, 3, 5, 2, 4)$.

Lo que NO es una poda

- «Si el objeto i pesa más que el promedio, no lo pongo.» Puede perder el óptimo. Es una **heurística**, no una poda.
- «En n reinas, la primera reina nunca va en la columna 1.» Falso para $n = 1$, y para $n = 4$: $(1, \dots)$ no da solución, pero para $n = 5$ sí: $(1, 3, 5, 2, 4)$.
- «Si ya encontré una solución válida, corto.» Es correcto para *decisión*; es **incorrecto** para optimización, conteo y enumeración.

Lo que NO es una poda

- «Si el objeto i pesa más que el promedio, no lo pongo.» Puede perder el óptimo. Es una **heurística**, no una poda.
- «En n reinas, la primera reina nunca va en la columna 1.» Falso para $n = 1$, y para $n = 4$: $(1, \dots)$ no da solución, pero para $n = 5$ sí: $(1, 3, 5, 2, 4)$.
- «Si ya encontré una solución válida, corto.» Es correcto para *decisión*; es **incorrecto** para optimización, conteo y enumeración.

Criterio

Una poda es una afirmación matemática sobre **todas** las extensiones de p . Si no la pueden demostrar, no es una poda: es una apuesta. Las apuestas también sirven (así juegan las computadoras al ajedrez), pero no resuelven el problema que les pidieron.

La idea: «esto no puede superar lo que ya tengo»

En optimización hay una segunda razón para no bajar por un subárbol: aunque tenga extensiones válidas, **ninguna es mejor que la mejor solución que ya encontramos.**

La idea: «esto no puede superar lo que ya tengo»

En optimización hay una segunda razón para no bajar por un subárbol: aunque tenga extensiones válidas, **ninguna es mejor que la mejor solución que ya encontramos**.

Ingredientes

- Una variable *mejor* con el valor de la mejor solución válida encontrada hasta el momento ($-\infty$ al principio).
- Una **cota superior** $U(p)$ tal que, para toda extensión válida a de p : $\text{valor}(a) \leq U(p)$.

Regla: si $U(p) \leq \text{mejor}$, no explorar el subárbol de p .

La idea: «esto no puede superar lo que ya tengo»

En optimización hay una segunda razón para no bajar por un subárbol: aunque tenga extensiones válidas, **ninguna es mejor que la mejor solución que ya encontramos**.

Ingredientes

- Una variable *mejor* con el valor de la mejor solución válida encontrada hasta el momento ($-\infty$ al principio).
- Una **cota superior** $U(p)$ tal que, para toda extensión válida a de p : $\text{valor}(a) \leq U(p)$.

Regla: si $U(p) \leq \text{mejor}$, no explorar el subárbol de p .

- Lo que hay que demostrar: *U es realmente una cota superior de todas las extensiones válidas de p .*

La idea: «esto no puede superar lo que ya tengo»

En optimización hay una segunda razón para no bajar por un subárbol: aunque tenga extensiones válidas, **ninguna es mejor que la mejor solución que ya encontramos**.

Ingredientes

- Una variable *mejor* con el valor de la mejor solución válida encontrada hasta el momento ($-\infty$ al principio).
- Una **cota superior** $U(p)$ tal que, para toda extensión válida a de p : $\text{valor}(a) \leq U(p)$.

Regla: si $U(p) \leq \text{mejor}$, no explorar el subárbol de p .

- Lo que hay que demostrar: *U es realmente una cota superior de todas las extensiones válidas de p .*
- Con eso: toda extensión válida de p vale a lo sumo $U(p) \leq \text{mejor}$, y *mejor* es el valor de una solución válida ya encontrada. Cortar no cambia el máximo.

La idea: «esto no puede superar lo que ya tengo»

En optimización hay una segunda razón para no bajar por un subárbol: aunque tenga extensiones válidas, **ninguna es mejor que la mejor solución que ya encontramos**.

Ingredientes

- Una variable *mejor* con el valor de la mejor solución válida encontrada hasta el momento ($-\infty$ al principio).
- Una **cota superior** $U(p)$ tal que, para toda extensión válida a de p : $\text{valor}(a) \leq U(p)$.

Regla: si $U(p) \leq \text{mejor}$, no explorar el subárbol de p .

- Lo que hay que demostrar: *U es realmente una cota superior de todas las extensiones válidas de p .*
- Con eso: toda extensión válida de p vale a lo sumo $U(p) \leq \text{mejor}$, y *mejor* es el valor de una solución válida ya encontrada. Cortar no cambia el máximo.
- Para minimizar: cota **inferior** $L(p)$, y se corta si $L(p) \geq \text{mejor}$.

Mochila: la cota «me llevo todo lo que queda»

Cota

$U(p) := va + \sum_{j \geq i} v_j$: el valor acumulado más el de **todos** los objetos que faltan decidir.

Mochila: la cota «me llevo todo lo que queda»

Cota

$U(p) := va + \sum_{j \geq i} v_j$: el valor acumulado más el de **todos** los objetos que faltan decidir.

- **Es cota superior:** si $a \in \text{Ext}(p)$, $\text{valor}(a) = va + \sum_{j \geq i} a_j v_j \leq va + \sum_{j \geq i} v_j$ porque $a_j \leq 1$ y $v_j \geq 0$. (Ni siquiera usamos que a es válida.)

Mochila: la cota «me llevo todo lo que queda»

Cota

$U(p) := va + \sum_{j \geq i} v_j$: el valor acumulado más el de **todos** los objetos que faltan decidir.

- **Es cota superior:** si $a \in \text{Ext}(p)$, $\text{valor}(a) = va + \sum_{j \geq i} a_j v_j \leq va + \sum_{j \geq i} v_j$ porque $a_j \leq 1$ y $v_j \geq 0$. (Ni siquiera usamos que a es válida.)
- Es una cota *floja*: ignora la capacidad. Una mejor: va más el óptimo de la mochila *fraccionaria* con lo que queda (se calcula en $O(n)$ si los objetos están ordenados por v_j/w_j). La demostración de que es cota es un lema de la clase de greedy.

Mochila: la cota «me llevo todo lo que queda»

Cota

$U(p) := va + \sum_{j \geq i} v_j$: el valor acumulado más el de **todos** los objetos que faltan decidir.

- **Es cota superior:** si $a \in \text{Ext}(p)$, $\text{valor}(a) = va + \sum_{j \geq i} a_j v_j \leq va + \sum_{j \geq i} v_j$ porque $a_j \leq 1$ y $v_j \geq 0$. (Ni siquiera usamos que a es válida.)
- Es una cota *floja*: ignora la capacidad. Una mejor: va más el óptimo de la mochila *fraccionaria* con lo que queda (se calcula en $O(n)$ si los objetos están ordenados por v_j/w_j). La demostración de que es cota es un lema de la clase de greedy.
- Cota más ajustada: más poda, pero más cuesta cada nodo. Otra vez un balance.

Mochila: la cota «me llevo todo lo que queda»

Cota

$U(p) := va + \sum_{j \geq i} v_j$: el valor acumulado más el de **todos** los objetos que faltan decidir.

- **Es cota superior:** si $a \in \text{Ext}(p)$, $\text{valor}(a) = va + \sum_{j \geq i} a_j v_j \leq va + \sum_{j \geq i} v_j$ porque $a_j \leq 1$ y $v_j \geq 0$. (Ni siquiera usamos que a es válida.)
- Es una cota *floja*: ignora la capacidad. Una mejor: va más el óptimo de la mochila *fraccionaria* con lo que queda (se calcula en $O(n)$ si los objetos están ordenados por v_j/w_j). La demostración de que es cota es un lema de la clase de greedy.
- Cota más ajustada: más poda, pero más cuesta cada nodo. Otra vez un balance.

Truco de implementación: precalcular $S_i := \sum_{j \geq i} v_j$ para todo i en $O(n)$; entonces $U(p) = va + S_i$ cuesta $O(1)$.

Mochila con las dos podas

Global *mejor*, inicializada en $-\infty$; $S_i = \sum_{j \geq i} v_j$ precalculado.

```
procedure mochila4(i, pa, va)  
  if pa > W then return                                // factibilidad;  
  if va +  $S_i \leq mejor$  then return                        // optimalidad;  
  if i = n + 1 then mejor  $\leftarrow va$ ; return              // hoja: va > mejor;  
  mochila4(i + 1, pa +  $w_i$ , va +  $v_i$ )                    // primero «lo pongo»;  
  mochila4(i + 1, pa, va);
```

Mochila con las dos podas

Global *mejor*, inicializada en $-\infty$; $S_i = \sum_{j \geq i} v_j$ precalculado.

```
procedure mochila4(i, pa, va)  
  if pa > W then return                                // factibilidad;  
  if va +  $S_i \leq mejor$  then return                        // optimalidad;  
  if i = n + 1 then mejor  $\leftarrow va$ ; return              // hoja: va > mejor;  
  mochila4(i + 1, pa + wi, va + vi)                  // primero «lo pongo»;  
  mochila4(i + 1, pa, va);
```

- Ya no devuelve un valor: **actualiza** *mejor*. Es lo natural cuando la poda depende de una variable global que cambia (alternativa: *mejor* por referencia).

Mochila con las dos podas

Global *mejor*, inicializada en $-\infty$; $S_i = \sum_{j \geq i} v_j$ precalculado.

```

procedure mochila4(i, pa, va)
  if pa > W then return                                // factibilidad;
  if va +  $S_i \leq mejor$  then return                      // optimalidad;
  if i = n + 1 then mejor ← va; return                  // hoja: va > mejor;
  mochila4(i + 1, pa + wi, va + vi)                // primero «lo pongo»;
  mochila4(i + 1, pa, va);

```

- Ya no devuelve un valor: **actualiza** *mejor*. Es lo natural cuando la poda depende de una variable global que cambia (alternativa: *mejor* por referencia).
- Al terminar mochila₄(1, 0, 0), *mejor* es el óptimo. Es más sutil que antes, porque *mejor cambia* durante el recorrido. Lo demostramos la clase que viene.

Asignación: una cota inferior no trivial

Minimizamos, así que necesitamos $L(p) \leq \text{costo}(\pi)$ para toda extensión π de $p = (\pi(1), \dots, \pi(i-1))$.

Asignación: una cota inferior no trivial

Minimizamos, así que necesitamos $L(p) \leq \text{costo}(\pi)$ para toda extensión π de $p = (\pi(1), \dots, \pi(i-1))$.

- $L_1(p) := ca$, el costo acumulado. Es cota porque los costos son ≥ 0 . Casi no poda.

Asignación: una cota inferior no trivial

Minimizamos, así que necesitamos $L(p) \leq \text{costo}(\pi)$ para toda extensión π de $p = (\pi(1), \dots, \pi(i-1))$.

- $L_1(p) := ca$, el costo acumulado. Es cota porque los costos son ≥ 0 . Casi no poda.
- $L_2(p) := ca + \sum_{k=i}^n \min_j c_{kj}$: cada persona que falta va a pagar *al menos* el mínimo de su fila.

Asignación: una cota inferior no trivial

Minimizamos, así que necesitamos $L(p) \leq \text{costo}(\pi)$ para toda extensión π de $p = (\pi(1), \dots, \pi(i-1))$.

- $L_1(p) := ca$, el costo acumulado. Es cota porque los costos son ≥ 0 . Casi no poda.
- $L_2(p) := ca + \sum_{k=i}^n \min_j c_{kj}$: cada persona que falta va a pagar *al menos* el mínimo de su fila.
- **Demostración de L_2 :** sea $\pi \in \text{Ext}(p)$. Entonces $\text{costo}(\pi) = ca + \sum_{k \geq i} c_{k\pi(k)}$, y para cada $k \geq i$ vale $c_{k\pi(k)} \geq \min_j c_{kj}$ por definición de mínimo. Sumando, $\text{costo}(\pi) \geq L_2(p)$. $\square$

Relajar para acotar

- L_2 ignora que las tareas no se pueden repetir: resuelve un problema **más fácil** cuyo óptimo es menor o igual que el del original. Eso es una **relajación**, y toda relajación da una cota.

Relajar para acotar

- L_2 ignora que las tareas no se pueden repetir: resuelve un problema **más fácil** cuyo óptimo es menor o igual que el del original. Eso es una **relajación**, y toda relajación da una cota.
- Se puede afinar: mínimo sólo sobre las tareas *no usadas* por p . ¿Por qué sigue siendo cota? (Toda extensión de p usa, para las personas $k \geq i$, tareas no usadas por p .)

Relajar para acotar

- L_2 ignora que las tareas no se pueden repetir: resuelve un problema **más fácil** cuyo óptimo es menor o igual que el del original. Eso es una **relajación**, y toda relajación da una cota.
- Se puede afinar: mínimo sólo sobre las tareas *no usadas* por p . ¿Por qué sigue siendo cota? (Toda extensión de p usa, para las personas $k \geq i$, tareas no usadas por p .)
- En mochila, la cota fraccionaria es también una relajación: permitimos $a_j \in [0, 1]$.

Relajar para acotar

- L_2 ignora que las tareas no se pueden repetir: resuelve un problema **más fácil** cuyo óptimo es menor o igual que el del original. Eso es una **relajación**, y toda relajación da una cota.
- Se puede afinar: mínimo sólo sobre las tareas *no usadas* por p . ¿Por qué sigue siendo cota? (Toda extensión de p usa, para las personas $k \geq i$, tareas no usadas por p .)
- En mochila, la cota fraccionaria es también una relajación: permitimos $a_j \in [0, 1]$.

→ Ejemplo completo de asignación en el pizarrón.

El orden de exploración importa

- Una poda por optimalidad compara contra *mejor*. Si $mejor = -\infty$, no corta nada. **Encontrar rápido una buena solución** hace que las podas actúen antes.

El orden de exploración importa

- Una poda por optimalidad compara contra *mejor*. Si $mejor = -\infty$, no corta nada. **Encontrar rápido una buena solución** hace que las podas actúen antes.
- Por eso en `mochila4` exploramos primero $a_i = 1$, y por eso conviene ordenar los objetos por v_j/w_j decreciente: las primeras hojas visitadas son «casi greedy».

El orden de exploración importa

- Una poda por optimalidad compara contra *mejor*. Si $mejor = -\infty$, no corta nada. **Encontrar rápido una buena solución** hace que las podas actúen antes.
- Por eso en mochila₄ exploramos primero $a_i = 1$, y por eso conviene ordenar los objetos por v_j/w_j decreciente: las primeras hojas visitadas son «casi greedy».
- También se puede inicializar *mejor* con el valor de *cualquier* solución válida (por ejemplo, la que da un greedy). Es correcto siempre que sea el valor de una solución válida real.

El orden de exploración importa

- Una poda por optimalidad compara contra *mejor*. Si $mejor = -\infty$, no corta nada. **Encontrar rápido una buena solución** hace que las podas actúen antes.
- Por eso en mochila₄ exploramos primero $a_i = 1$, y por eso conviene ordenar los objetos por v_j/w_j decreciente: las primeras hojas visitadas son «casi greedy».
- También se puede inicializar *mejor* con el valor de *cualquier* solución válida (por ejemplo, la que da un greedy). Es correcto siempre que sea el valor de una solución válida real.
- Nada de esto cambia el peor caso; cambia (mucho) el caso típico. Fuerza bruta + cotas + buen orden se llama **branch and bound**.

El orden de exploración importa

- Una poda por optimalidad compara contra *mejor*. Si $mejor = -\infty$, no corta nada. **Encontrar rápido una buena solución** hace que las podas actúen antes.
- Por eso en mochila₄ exploramos primero $a_i = 1$, y por eso conviene ordenar los objetos por v_j/w_j decreciente: las primeras hojas visitadas son «casi greedy».
- También se puede inicializar *mejor* con el valor de *cualquier* solución válida (por ejemplo, la que da un greedy). Es correcto siempre que sea el valor de una solución válida real.
- Nada de esto cambia el peor caso; cambia (mucho) el caso típico. Fuerza bruta + cotas + buen orden se llama **branch and bound**.

Para la práctica

Se les va a pedir: proponer la poda, **demostrar** que es correcta, e implementarla. No se les va a pedir que demuestren que *ayuda*: eso se mide.

Decisión: ¿existe?

Para n reinas con $Sols = \{1, \dots, n\}^n$ y la poda R_2 (no se atacan):

```
function reinas( $i, p$ )  
  if  $\neg R_2(p)$  then return False    // la reina  $i-1$  ataca a otra;  
  if  $i = n + 1$  then return True;  
  for  $j \leftarrow 1$  to  $n$  do  
    if reinas( $i + 1, p \oplus j$ ) then return True;  
  return False ;
```

Decisión: ¿existe?

Para n reinas con $Sols = \{1, \dots, n\}^n$ y la poda R_2 (no se atacan):

```
function reinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return False    // la reina  $i-1$  ataca a otra;  
  if  $i = n + 1$  then return True;  
  for  $j \leftarrow 1$  to  $n$  do  
    if reinas( $i + 1$ ,  $p \oplus j$ ) then return True;  
  return False ;
```

- Combinamos con $\vee$ en vez de máx ; el neutro es False en vez de $-\infty$.

Decisión: ¿existe?

Para n reinas con $Sols = \{1, \dots, n\}^n$ y la poda R_2 (no se atacan):

```
function reinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return False    // la reina  $i-1$  ataca a otra;  
  if  $i = n + 1$  then return True;  
  for  $j \leftarrow 1$  to  $n$  do  
    if reinas( $i + 1$ ,  $p \oplus j$ ) then return True;  
  return False ;
```

- Combinamos con $\vee$ en vez de máx ; el neutro es False en vez de $-\infty$.
- El *return* temprano en el for es una poda gratis: si una rama da True, las otras no cambian el resultado. **Sólo vale para decisión.**

Decisión: ¿existe?

Para n reinas con $Sols = \{1, \dots, n\}^n$ y la poda R_2 (no se atacan):

```
function reinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return False    // la reina  $i-1$  ataca a otra;  
  if  $i = n + 1$  then return True;  
  for  $j \leftarrow 1$  to  $n$  do  
    if reinas( $i + 1$ ,  $p \oplus j$ ) then return True;  
  return False ;
```

- Combinamos con $\vee$ en vez de máx ; el neutro es False en vez de $-\infty$.
- El *return* temprano en el for es una poda gratis: si una rama da True, las otras no cambian el resultado. **Sólo vale para decisión.**
- Chequear $R_2(p)$ cuesta $O(n)$ si sólo comparamos la última reina contra las anteriores.

Construcción: devolver la solución

Para **devolver** una solución hay que *tener* la parcial. Globales: *mejor*, *sol*[1..*n*] (la parcial actual) y *mejorSol*.

```
procedure mochila5(i, pa, va)  
  if pa > W or va + Si ≤ mejor then return;  
  if i = n + 1 then mejor ← va ;  
  mejorSol ← copia de sol ;  
  return;  
  sol[i] ← 1;   mochila5(i + 1, pa + wi, va + vi) ;  
  sol[i] ← 0;   mochila5(i + 1, pa, va) ;
```

Construcción: devolver la solución

Para **devolver** una solución hay que *tener* la parcial. Globales: *mejor*, *sol*[1..*n*] (la parcial actual) y *mejorSol*.

```
procedure mochila5(i, pa, va)  
  if pa > W or va + Si ≤ mejor then return;  
  if i = n + 1 then mejor ← va ;  
  mejorSol ← copia de sol ;  
  return;  
  sol[i] ← 1;   mochila5(i + 1, pa + wi, va + vi) ;  
  sol[i] ← 0;   mochila5(i + 1, pa, va) ;
```

- *sol*[1..*i* − 1] **es** la parcial *p*: la *compartimos* y la deshacemos al volver (eso es literalmente el *backtrack*). Copiar *mejorSol* cuesta $O(n)$, sólo al mejorar.

Construcción: devolver la solución

Para **devolver** una solución hay que *tener* la parcial. Globales: *mejor*, *sol*[1..*n*] (la parcial actual) y *mejorSol*.

```

procedure mochila5(i, pa, va)
  if pa > W or va + Si ≤ mejor then return;
  if i = n + 1 then mejor ← va; ;
  mejorSol ← copia de sol; ;
  return;
  sol[i] ← 1;  mochila5(i + 1, pa + wi, va + vi) ;
  sol[i] ← 0;  mochila5(i + 1, pa, va) ;
  
```

- *sol*[1..*i* − 1] **es** la parcial *p*: la *compartimos* y la deshacemos al volver (eso es literalmente el *backtrack*). Copiar *mejorSol* cuesta $O(n)$, sólo al mejorar.
- Si pasáramos $p \oplus 0$ por copia, cada nodo costaría $O(n)$. Hay que decidirlo y decirlo.

Enumeración: todas las soluciones

- Mismo recorrido; en cada hoja válida, **imprimir** y **seguir**. Nada de *return* temprano.

Enumeración: todas las soluciones

- Mismo recorrido; en cada hoja válida, **imprimir** y **seguir**. Nada de *return* temprano.
- Las podas por factibilidad siguen valiendo tal cual (un subárbol sin válidas no aporta nada a la lista).

Enumeración: todas las soluciones

- Mismo recorrido; en cada hoja válida, **imprimir** y **seguir**. Nada de *return* temprano.
- Las podas por factibilidad siguen valiendo tal cual (un subárbol sin válidas no aporta nada a la lista).
- Las podas por optimalidad no aplican (no hay «mejor»). Salvo que pidan *todas las óptimas*: entonces sí, pero cortando con $U(p) < \text{mejor}$ **estricto** (¿por qué?), y vaciando la lista cada vez que *mejor* mejora.

Enumeración: todas las soluciones

- Mismo recorrido; en cada hoja válida, **imprimir** y **seguir**. Nada de *return* temprano.
- Las podas por factibilidad siguen valiendo tal cual (un subárbol sin válidas no aporta nada a la lista).
- Las podas por optimalidad no aplican (no hay «mejor»). Salvo que pidan *todas las óptimas*: entonces sí, pero cortando con $U(p) < \text{mejor estricto}$ (¿por qué?), y vaciando la lista cada vez que *mejor* mejora.
- Cota inferior de complejidad: hay que escribir cada solución, así que $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Para 8 reinas: $92 \cdot 8$; para subconjuntos de un conjunto: puede ser $\Theta(n \cdot 2^n)$ aunque el árbol se pode muchísimo.

Enumeración: todas las soluciones

- Mismo recorrido; en cada hoja válida, **imprimir** y **seguir**. Nada de *return* temprano.
- Las podas por factibilidad siguen valiendo tal cual (un subárbol sin válidas no aporta nada a la lista).
- Las podas por optimalidad no aplican (no hay «mejor»). Salvo que pidan *todas las óptimas*: entonces sí, pero cortando con $U(p) < \text{mejor estricto}$ (¿por qué?), y vaciando la lista cada vez que *mejor* mejora.
- Cota inferior de complejidad: hay que escribir cada solución, así que $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Para 8 reinas: $92 \cdot 8$; para subconjuntos de un conjunto: puede ser $\Theta(n \cdot 2^n)$ aunque el árbol se pode muchísimo.

Si les piden enumerar, la complejidad va a depender de *cuántas* válidas hay, y eso a veces es un problema de conteo difícil en sí mismo. Alcanza con dar la cota del tamaño del árbol.

Conteo: ¿cuántas?

```
function contarReinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return 0;  
  if  $i = n + 1$  then return 1;  
   $c \leftarrow 0$  ;  
  for  $j \leftarrow 1$  to  $n$  do  $c \leftarrow c + \text{contarReinas}(i + 1, p \oplus j)$ ;  
  return  $c$  ;
```

Conteo: ¿cuántas?

```
function contarReinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return 0;  
  if  $i = n + 1$  then return 1;  
   $c \leftarrow 0$  ;  
  for  $j \leftarrow 1$  to  $n$  do  $c \leftarrow c + \text{contarReinas}(i + 1, p \oplus j)$ ;  
  return  $c$  ;
```

- Combinamos con $+$; neutro 0; hoja válida vale 1. $\text{contarReinas}(1, ())$ para $n = 8$ devuelve 92.

Conteo: ¿cuántas?

```
function contarReinas( $i, p$ )  
  if  $\neg R_2(p)$  then return 0;  
  if  $i = n + 1$  then return 1;  
   $c \leftarrow 0$  ;  
  for  $j \leftarrow 1$  to  $n$  do  $c \leftarrow c + \text{contarReinas}(i + 1, p \oplus j)$ ;  
  return  $c$  ;
```

- Combinamos con $+$; neutro 0; hoja válida vale 1. $\text{contarReinas}(1, ())$ para $n = 8$ devuelve 92.
- **Semántica** de la función: $\text{contarReinas}(i, p) = |\text{Ext}(p) \cap \text{Sols}_{\text{válidas}}|$, la cantidad de extensiones válidas de p . Con eso, la corrección de la poda dice: si $\neg R_2(p)$ entonces ese conjunto es vacío, y 0 es la respuesta correcta.

Conteo: ¿cuántas?

```
function contarReinas( $i$ ,  $p$ )  
  if  $\neg R_2(p)$  then return 0;  
  if  $i = n + 1$  then return 1;  
   $c \leftarrow 0$  ;  
  for  $j \leftarrow 1$  to  $n$  do  $c \leftarrow c + \text{contarReinas}(i + 1, p \oplus j)$ ;  
  return  $c$  ;
```

- Combinamos con $+$; neutro 0; hoja válida vale 1. $\text{contarReinas}(1, ())$ para $n = 8$ devuelve 92.
- **Semántica** de la función: $\text{contarReinas}(i, p) = |\text{Ext}(p) \cap \text{Sols}_{\text{válidas}}|$, la cantidad de extensiones válidas de p . Con eso, la corrección de la poda dice: si $\neg R_2(p)$ entonces ese conjunto es vacío, y 0 es la respuesta correcta.
- Con $\text{Sols} = \text{permutaciones}$ y sin R_2 : 40 320 hojas visitadas. Con R_2 : unos 2 000 nodos.

Resumen: la misma recursión, cambiando tres cosas

Pregunta	Hoja válida / no válida	Combinar hijos	Corte temprano	Podas
Decisión	True / False	✓	sí, al primer True	factibilidad
Construcción	guardar p / nada	✓	sí	factibilidad
Conteo	1 / 0	+	no	factibilidad
Enumeración	imprimir p / nada	(nada)	no	factibilidad
Optimización	valor(p) / $-\infty$	máx	no	factibilidad y optimalidad

Resumen: la misma recursión, cambiando tres cosas

Pregunta	Hoja válida / no válida	Combinar hijos	Corte temprano	Podas
Decisión	True / False	✓	sí, al primer True	factibilidad
Construcción	guardar p / nada	✓	sí	factibilidad
Conteo	1 / 0	+	no	factibilidad
Enumeración	imprimir p / nada	(nada)	no	factibilidad
Optimización	valor(p) / $-\infty$	máx	no	factibilidad y optimalidad

- La **formalización** (Sols, válida, parciales, sucesoras) es la misma en las cinco filas. Por eso la escribimos primero y una sola vez.

Resumen: la misma recursión, cambiando tres cosas

Pregunta	Hoja válida / no válida	Combinar hijos	Corte temprano	Podas
Decisión	True / False	✓	sí, al primer True	factibilidad
Construcción	guardar p / nada	✓	sí	factibilidad
Conteo	1 / 0	+	no	factibilidad
Enumeración	imprimir p / nada	(nada)	no	factibilidad
Optimización	valor(p) / $-\infty$	máx	no	factibilidad y optimalidad

- La **formalización** (Sols, válida, parciales, sucesoras) es la misma en las cinco filas. Por eso la escribimos primero y una sola vez.
- Erickson recomienda diseñar siempre la versión más simple (decisión, o el valor óptimo) y después adaptarla. Nosotros también.

La cuenta básica

Un algoritmo de backtracking recorre un árbol. Su tiempo es

$$T = \sum_{\text{nodos visitados } p} (\text{trabajo en } p, \text{ sin contar las llamadas recursivas}).$$

La cuenta básica

Un algoritmo de backtracking recorre un árbol. Su tiempo es

$$T = \sum_{\text{nodos visitados } p} (\text{trabajo en } p, \text{ sin contar las llamadas recursivas}).$$

- Cota superior cómoda:

$$T \leq (\# \text{nodos del árbol completo}) \times (\text{máximo trabajo por nodo}).$$

La cuenta básica

Un algoritmo de backtracking recorre un árbol. Su tiempo es

$$T = \sum_{\text{nodos visitados } p} (\text{trabajo en } p, \text{ sin contar las llamadas recursivas}).$$

- Cota superior cómoda:

$$T \leq (\# \text{nodos del árbol completo}) \times (\text{máximo trabajo por nodo}).$$

- Un árbol donde cada nodo tiene a lo sumo k hijos y de altura n tiene a lo sumo

$$1 + k + k^2 + \dots + k^n = \frac{k^{n+1} - 1}{k - 1} = O(k^n) \quad \text{nodos} \quad (2^{n+1} - 1 \text{ si } k = 2).$$

La cuenta básica

Un algoritmo de backtracking recorre un árbol. Su tiempo es

$$T = \sum_{\text{nodos visitados } p} (\text{trabajo en } p, \text{ sin contar las llamadas recursivas}).$$

- Cota superior cómoda:

$$T \leq (\# \text{nodos del árbol completo}) \times (\text{máximo trabajo por nodo}).$$

- Un árbol donde cada nodo tiene a lo sumo k hijos y de altura n tiene a lo sumo

$$1 + k + k^2 + \dots + k^n = \frac{k^{n+1} - 1}{k - 1} = O(k^n) \quad \text{nodos} \quad (2^{n+1} - 1 \text{ si } k = 2).$$

- Con podas el árbol visitado es un *subárbol* del completo, así que la misma cota sigue valiendo. Y en el peor caso, en general, se alcanza.

La cuenta básica

Un algoritmo de backtracking recorre un árbol. Su tiempo es

$$T = \sum_{\text{nodos visitados } p} (\text{trabajo en } p, \text{ sin contar las llamadas recursivas}).$$

- Cota superior cómoda:

$$T \leq (\# \text{nodos del árbol completo}) \times (\text{máximo trabajo por nodo}).$$

- Un árbol donde cada nodo tiene a lo sumo k hijos y de altura n tiene a lo sumo

$$1 + k + k^2 + \dots + k^n = \frac{k^{n+1} - 1}{k - 1} = O(k^{n+1}) \quad \text{nodos} \quad (2^{n+1} - 1 \text{ si } k = 2).$$

- Con podas el árbol visitado es un *subárbol* del completo, así que la misma cota sigue valiendo. Y en el peor caso, en general, se alcanza.

Nivel de rigor esperado: alcanza con «árbol binario de altura n , $O(2^n)$ nodos, $O(1)$ por nodo». Sí hay que decir **cuánto cuesta cada nodo** y de qué depende.

Mochila: las cuatro versiones

Versión	Nodos	Trabajo por nodo	Tiempo
$\text{mochila}_1(i, p)$, copia p	$2^{n+1} - 1$	$O(n)$ (copiar, sumar en la hoja)	$O(n \cdot 2^n)$
$\text{mochila}_2(i, pa, va)$	$2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_3 (factibilidad)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_4 (ambas, S_i precalculado)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$

Mochila: las cuatro versiones

Versión	Nodos	Trabajo por nodo	Tiempo
$\text{mochila}_1(i, p)$, copia p	$2^{n+1} - 1$	$O(n)$ (copiar, sumar en la hoja)	$O(n \cdot 2^n)$
$\text{mochila}_2(i, pa, va)$	$2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_3 (factibilidad)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_4 (ambas, S_i precalculado)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$

- Otra forma de ver mochila_2 : $T(m) = 2 T(m - 1) + O(1)$ con $m = n - i + 1$ objetos por decidir. Se resuelve como en la clase de repaso: $T(m) = O(2^m)$.

Mochila: las cuatro versiones

Versión	Nodos	Trabajo por nodo	Tiempo
$\text{mochila}_1(i, p)$, copia p	$2^{n+1} - 1$	$O(n)$ (copiar, sumar en la hoja)	$O(n \cdot 2^n)$
$\text{mochila}_2(i, pa, va)$	$2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_3 (factibilidad)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_4 (ambas, S_i precalculado)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$

- Otra forma de ver mochila_2 : $T(m) = 2 T(m - 1) + O(1)$ con $m = n - i + 1$ objetos por decidir. Se resuelve como en la clase de repaso: $T(m) = O(2^m)$.
- ¿Las podas mejoran el peor caso? **No**: con $W \geq \sum w_j$ y *mejor* que crece de a poco (por ejemplo, $v = (1, 1, \dots, 1)$ explorando primero $a_i = 0$), se visita todo. Es un buen ejercicio construir la familia de instancias.

Mochila: las cuatro versiones

Versión	Nodos	Trabajo por nodo	Tiempo
$\text{mochila}_1(i, p)$, copia p	$2^{n+1} - 1$	$O(n)$ (copiar, sumar en la hoja)	$O(n \cdot 2^n)$
$\text{mochila}_2(i, pa, va)$	$2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_3 (factibilidad)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$
mochila_4 (ambas, S_i precalculado)	$\leq 2^{n+1} - 1$	$O(1)$	$O(2^n)$

- Otra forma de ver mochila_2 : $T(m) = 2 T(m - 1) + O(1)$ con $m = n - i + 1$ objetos por decidir. Se resuelve como en la clase de repaso: $T(m) = O(2^m)$.
- ¿Las podas mejoran el peor caso? **No**: con $W \geq \sum w_j$ y *mejor* que crece de a poco (por ejemplo, $v = (1, 1, \dots, 1)$ explorando primero $a_i = 0$), se visita todo. Es un buen ejercicio construir la familia de instancias.
- **Espacio**: profundidad de recursión n , cada llamada usa $O(1)$: $O(n)$. Si guardamos *sol* y *mejorSol*: $O(n)$ más. Si *copiamos* p en cada llamada: hasta n copias vivas de longitud $\leq n$, $O(n^2)$.

n reinas: el universo importa (otra vez)

- Con $\text{Sols} = \{1, \dots, n\}^n$: cada nodo tiene n hijos, altura n : $\frac{n^{n+1} - 1}{n - 1} = O(n^n)$ nodos.
Chequear la reina nueva contra las anteriores: $O(n)$. Total: $O(n \cdot n^n) = O(n^{n+1})$.

n reinas: el universo importa (otra vez)

- Con $\text{Sols} = \{1, \dots, n\}^n$: cada nodo tiene n hijos, altura n : $\frac{n^{n+1} - 1}{n - 1} = O(n^n)$ nodos.
Chequear la reina nueva contra las anteriores: $O(n)$. Total: $O(n \cdot n^n) = O(n^{n+1})$.
- Con $\text{Sols} = \text{permutaciones}$: en el nivel i hay $n(n-1) \cdots (n-i+1) = \frac{n!}{(n-i)!}$ nodos.

Sumando:

$$\sum_{i=0}^n \frac{n!}{(n-i)!} = n! \sum_{k=0}^n \frac{1}{k!} \leq e \cdot n!,$$

o sea $O(n!)$ nodos y $O(n \cdot n!)$ de tiempo.

n reinas: el universo importa (otra vez)

- Con $\text{Sols} = \{1, \dots, n\}^n$: cada nodo tiene n hijos, altura n : $\frac{n^{n+1} - 1}{n - 1} = O(n^n)$ nodos.
Chequear la reina nueva contra las anteriores: $O(n)$. Total: $O(n \cdot n^n) = O(n^{n+1})$.
- Con $\text{Sols} = \text{permutaciones}$: en el nivel i hay $n(n-1) \cdots (n-i+1) = \frac{n!}{(n-i)!}$ nodos.

Sumando:

$$\sum_{i=0}^n \frac{n!}{(n-i)!} = n! \sum_{k=0}^n \frac{1}{k!} \leq e \cdot n!,$$

o sea $O(n!)$ nodos y $O(n \cdot n!)$ de tiempo.

- n^n contra $n!$: por Stirling, $n! \approx (n/e)^n$, una ganancia de e^n . Para $n = 12$: $8,9 \times 10^{12}$ contra $4,8 \times 10^8$.

n reinas: el universo importa (otra vez)

- Con $\text{Sols} = \{1, \dots, n\}^n$: cada nodo tiene n hijos, altura n : $\frac{n^{n+1} - 1}{n - 1} = O(n^n)$ nodos.
Chequear la reina nueva contra las anteriores: $O(n)$. Total: $O(n \cdot n^n) = O(n^{n+1})$.
- Con $\text{Sols} = \text{permutaciones}$: en el nivel i hay $n(n-1) \cdots (n-i+1) = \frac{n!}{(n-i)!}$ nodos.
Sumando:

$$\sum_{i=0}^n \frac{n!}{(n-i)!} = n! \sum_{k=0}^n \frac{1}{k!} \leq e \cdot n!,$$

o sea $O(n!)$ nodos y $O(n \cdot n!)$ de tiempo.

- n^n contra $n!$: por Stirling, $n! \approx (n/e)^n$, una ganancia de e^n . Para $n = 12$: $8,9 \times 10^{12}$ contra $4,8 \times 10^8$.

Lo mismo aplica a asignación: $O(n!)$ nodos, $O(1)$ por nodo si los mínimos de fila están precalculados y «marcar tarea usada» es $O(1)$.

¿Hasta dónde llega la fuerza bruta?

Suponiendo 10^8 operaciones simples por segundo:

Nodos	$n = 10$	$n = 20$	$n = 30$
2^n	10^{-5} s	0,01 s	11 s
$n^2 \cdot 2^n$	10^{-3} s	4 s	2,7 hs
$n!$	0,04 s	770 años	8×10^{16} años
n^n	100 s	3×10^{10} años	...

¿Hasta dónde llega la fuerza bruta?

Suponiendo 10^8 operaciones simples por segundo:

Nodos	$n = 10$	$n = 20$	$n = 30$
2^n	10^{-5} s	0,01 s	11 s
$n^2 \cdot 2^n$	10^{-3} s	4 s	2,7 hs
$n!$	0,04 s	770 años	8×10^{16} años
n^n	100 s	3×10^{10} años	...

- 2^n es «razonable» hasta $n \approx 30$; $n!$ hasta $n \approx 12$; n^n hasta $n \approx 9$. Las podas mueven estos límites en la práctica, a veces muchísimo, pero no cambian las columnas.

¿Hasta dónde llega la fuerza bruta?

Suponiendo 10^8 operaciones simples por segundo:

Nodos	$n = 10$	$n = 20$	$n = 30$
2^n	10^{-5} s	0,01 s	11 s
$n^2 \cdot 2^n$	10^{-3} s	4 s	2,7 hs
$n!$	0,04 s	770 años	8×10^{16} años
n^n	100 s	3×10^{10} años	...

- 2^n es «razonable» hasta $n \approx 30$; $n!$ hasta $n \approx 12$; n^n hasta $n \approx 9$. Las podas mueven estos límites en la práctica, a veces muchísimo, pero no cambian las columnas.
- Cuando el problema admite algo mejor (programación dinámica: mochila en $O(nW)$), la fuerza bruta queda como oráculo de tests para n chico.

Mochila en C++, con podas, construyendo la solución

```
int n, W; vector<int> w, v, S;          //  $S[i] = v[i] + \dots + v[n-1]$ ,  $S[n] = 0$ 
int mejor = -1; vector<int> sol, mejorSol;

void mochila(int i, int pa, int va) {
    if (pa > W) return;                  // poda por factibilidad
    if (va + S[i] <= mejor) return;      // poda por optimalidad
    if (i == n) { mejor = va; mejorSol = sol; return; }
    sol[i] = 1; mochila(i + 1, pa + w[i], va + v[i]);
    sol[i] = 0; mochila(i + 1, pa, va);
}
```

Mochila en C++, con podas, construyendo la solución

```
int n, W; vector<int> w, v, S;          //  $S[i] = v[i] + \dots + v[n-1]$ ,  $S[n] = 0$ 
int mejor = -1; vector<int> sol, mejorSol;

void mochila(int i, int pa, int va) {
    if (pa > W) return;                  // poda por factibilidad
    if (va + S[i] <= mejor) return;      // poda por optimalidad
    if (i == n) { mejor = va; mejorSol = sol; return; }
    sol[i] = 1; mochila(i + 1, pa + w[i], va + v[i]);
    sol[i] = 0; mochila(i + 1, pa, va);
}
```

- $mejor = -1$ hace de $-\infty$: cualquier solución válida vale ≥ 0 . Hay que **decirlo**.

Mochila en C++, con podas, construyendo la solución

```
int n, W; vector<int> w, v, S;          //  $S[i] = v[i] + \dots + v[n-1]$ ,  $S[n] = 0$ 
int mejor = -1; vector<int> sol, mejorSol;

void mochila(int i, int pa, int va) {
    if (pa > W) return;                  // poda por factibilidad
    if (va + S[i] <= mejor) return;      // poda por optimalidad
    if (i == n) { mejor = va; mejorSol = sol; return; }
    sol[i] = 1; mochila(i + 1, pa + w[i], va + v[i]);
    sol[i] = 0; mochila(i + 1, pa, va);
}
```

- $mejor = -1$ hace de $-\infty$: cualquier solución válida vale ≥ 0 . Hay que **decirlo**.
- Índices desde 0 y caso base $i = n$: la traducción de la fórmula (con i desde 1 y base $n + 1$) al código es directa, y no se demuestra.

Mochila en C++, con podas, construyendo la solución

```
int n, W; vector<int> w, v, S;          //  $S[i] = v[i] + \dots + v[n-1]$ ,  $S[n] = 0$ 
int mejor = -1; vector<int> sol, mejorSol;

void mochila(int i, int pa, int va) {
    if (pa > W) return;                  // poda por factibilidad
    if (va + S[i] <= mejor) return;      // poda por optimalidad
    if (i == n) { mejor = va; mejorSol = sol; return; }
    sol[i] = 1; mochila(i + 1, pa + w[i], va + v[i]);
    sol[i] = 0; mochila(i + 1, pa, va);
}
```

- $mejor = -1$ hace de $-\infty$: cualquier solución válida vale ≥ 0 . Hay que **decirlo**.
- Índices desde 0 y caso base $i = n$: la traducción de la fórmula (con i desde 1 y base $n + 1$) al código es directa, y no se demuestra.
- **Testear** contra $mochila_1$ (sin podas) en instancias al azar con $n \leq 15$: si difieren, alguna poda está mal (o su demostración).

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.
- 3 **Parciales y sucesoras:** qué es una decisión, qué decisiones son legales, qué *resumen* de las decisiones pasadas necesita la recursión.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.
- 3 **Parciales y sucesoras:** qué es una decisión, qué decisiones son legales, qué *resumen* de las decisiones pasadas necesita la recursión.
- 4 **Función recursiva:** caso base (hoja: ¿válida? ¿cuánto vale?), caso recursivo (combinar sucesoras con $\vee$, $+$, máx). Escribir su **semántica**.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.
- 3 **Parciales y sucesoras:** qué es una decisión, qué decisiones son legales, qué *resumen* de las decisiones pasadas necesita la recursión.
- 4 **Función recursiva:** caso base (hoja: ¿válida? ¿cuánto vale?), caso recursivo (combinar sucesoras con $\vee$, $+$, máx). Escribir su **semántica**.
- 5 **Podas:** factibilidad (condición necesaria) y optimalidad (cota), *con demostración*.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.
- 3 **Parciales y sucesoras:** qué es una decisión, qué decisiones son legales, qué *resumen* de las decisiones pasadas necesita la recursión.
- 4 **Función recursiva:** caso base (hoja: ¿válida? ¿cuánto vale?), caso recursivo (combinar sucesoras con $\vee$, $+$, máx). Escribir su **semántica**.
- 5 **Podas:** factibilidad (condición necesaria) y optimalidad (cota), *con demostración*.
- 6 **Complejidad:** nodos del árbol $\times$ trabajo por nodo; espacio: profundidad $\times$ estado.

La receta de backtracking

Frente a un enunciado nuevo, en este orden:

- 1 **Candidatas:** elegir Sols (vectores de qué, con qué valores) y escribir la semántica: qué objeto del enunciado representa cada vector.
- 2 **Válidas:** escribir válida(a) como fórmula. Si es optimización: valor(a) y el orden.
- 3 **Parciales y sucesoras:** qué es una decisión, qué decisiones son legales, qué *resumen* de las decisiones pasadas necesita la recursión.
- 4 **Función recursiva:** caso base (hoja: ¿válida? ¿cuánto vale?), caso recursivo (combinar sucesoras con $\vee$, $+$, máx). Escribir su **semántica**.
- 5 **Podas:** factibilidad (condición necesaria) y optimalidad (cota), *con demostración*.
- 6 **Complejidad:** nodos del árbol $\times$ trabajo por nodo; espacio: profundidad $\times$ estado.
- 7 **Implementar**, y testear contra la versión sin podas.

Tips

- 1 Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.

Tips

- 1 Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.
- 2 Diseñen la versión más simple (decisión o valor óptimo). Construir, contar y enumerar son modificaciones de cinco líneas.

Tips

- 1 Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.
- 2 Diseñen la versión más simple (decisión o valor óptimo). Construir, contar y enumerar son modificaciones de cinco líneas.
- 3 Los parámetros de la recursión son el *resumen* de las decisiones pasadas. Si necesitan la parcial entera, compártanla y deshagan al volver.

Tips

- 1 Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.
- 2 Diseñen la versión más simple (decisión o valor óptimo). Construir, contar y enumerar son modificaciones de cinco líneas.
- 3 Los parámetros de la recursión son el *resumen* de las decisiones pasadas. Si necesitan la parcial entera, compártanla y deshagan al volver.
- 4 Toda poda viene con una demostración de una línea que empieza «sea a una extensión cualquiera de p » y termina «luego a no es válida» (o «luego $\text{valor}(a) \leq U(p)$ »).

Tips

- ➊ Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.
- ➋ Diseñen la versión más simple (decisión o valor óptimo). Construir, contar y enumerar son modificaciones de cinco líneas.
- ➌ Los parámetros de la recursión son el *resumen* de las decisiones pasadas. Si necesitan la parcial entera, compártanla y deshagan al volver.
- ➍ Toda poda viene con una demostración de una línea que empieza «sea a una extensión cualquiera de p » y termina «luego a no es válida» (o «luego $\text{valor}(a) \leq U(p)$ »).
- ➎ El peor caso con podas es, en general, el mismo que sin podas. No prometan más de lo que pueden demostrar.

Tips

- ➊ Antes de programar, escriban Sols y válida en un papel. Si no pueden, no entienden el problema todavía.
- ➋ Diseñen la versión más simple (decisión o valor óptimo). Construir, contar y enumerar son modificaciones de cinco líneas.
- ➌ Los parámetros de la recursión son el *resumen* de las decisiones pasadas. Si necesitan la parcial entera, compártanla y deshagan al volver.
- ➍ Toda poda viene con una demostración de una línea que empieza «sea a una extensión cualquiera de p » y termina «luego a no es válida» (o «luego $\text{valor}(a) \leq U(p)$ »).
- ➎ El peor caso con podas es, en general, el mismo que sin podas. No prometan más de lo que pueden demostrar.
- ➏ Testeen contra la fuerza bruta pura. Es el mejor test que van a tener en toda la materia.

Bibliografía

- J. Erickson, *Algorithms*, cap. 2 (Backtracking). Disponible libremente en jeffe.cs.illinois.edu/teaching/algorithms. Es la referencia principal de esta clase.
- G. Brassard, P. Bratley, *Fundamentals of Algorithmics*, cap. 9 (Exploring graphs: backtracking, branch-and-bound).
- S. Skiena, *The Algorithm Design Manual*, cap. 9 (Combinatorial search).
- T. Cormen, C. Leiserson, R. Rivest, C. Stein, *Introduction to Algorithms*, cap. 4 (recurrencias) y apéndice C (conteo).
- J. Kleinberg, É. Tardos, *Algorithm Design*, cap. 2 (fundamentos) y cap. 10 (búsqueda exhaustiva acotada en problemas difíciles).

¿Preguntas?